

CS336 Assignment 5 Writeup

各题题干均原样摘自 cs336_spring2026_assignment5_alignment.pdf (英文正文, 不用代码块包裹, 以便公式正常渲染) GPU 的书面题、已通过单元测试的代码题, 以及第四至六章的实验已填写。第七章要求自己提出一个 policy-gradient estimator, 仍待填写。

提交: writeup.pdf, 以及用 test_and_make_submission.sh 生成的 code.zip。

3 Prompting

3.4 实验

题目 (prompting_baselines): 在 GSM8K 上运行 OLMo-2-0425-1B (5 分) 题目来源: cs336_spring2026_assignment5

Run OLMo-2-0425-1B on GSM8K (5 points)

- (a) Write a script to evaluate OLMo-2-0425-1B performance on GSM8K with zero-shot question_only, zero-shot r1_zero, and few-shot r1_zero_three_shot prompts.

Then, run your script and observe the outputs. For each prompt, how many model generations fall into each of the following categories: (1) correct with both format and correctness reward 1, (2) format reward 1 and correctness reward 0, (3) format reward 0 and correctness reward 0? Observing at least ten examples of category 2, how many model outputs are actually correct but just not parsed properly? What about category 3?

Deliverable: A few sentences of commentary, the evaluation metrics, and a few examples of prompts and responses.

- (b) Observing the model outputs, characterize the model's behavior with each prompt. For example, if we want the model to answer the question, is it enough to just provide the question, or does the model exhibit other behaviors besides just answering the question? How do the zero-shot r1_zero and few-shot r1_zero_three_shot prompts shape the model's behavior?

Deliverable: A few sentences of commentary with supporting examples.

答案

2026-09-29, AutoDL 1x RTX PRO 6000, 本地权重 /root/autodl-tmp/models/OLMo-2-0425-1B。脚本 cs336_alignment/prompting_baselines.py, GSM8K 测试集 1319 题。vLLM 请求显式传 temperature=1.0、top_p=1.0 (与 vLLM 默认 top-p 一致), max_tokens=512。r1_zero 与 r1_zero_three_shot 在 </answer> 停止并保留该串; question_only 不设这个停止串。

(a)

prompt	格式对且答案对	格式对、答案错	格式错
question_only	2 (0.2%)	292 (22.1%)	1025 (77.7%)
r1_zero	1 (0.1%)	697 (52.8%)	621 (47.1%)
r1_zero_three_shot	267 (20.2%)	978 (74.1%)	74 (5.6%)

上表是测试集脚本打印的三类计数。prompting_baselines.py 不保存逐条输出, 原来那次「各看 10 条」没有留在仓库里, 下面不再引用那次抽查的比例。例子改自基座模型、训练前的验证集文件 val_rollouts_step_-1.jsonl (seed 0, 各 1024 条), reward 用文件里记下的分数。

实际写进 prompt 的 question_only 模板是 {question} Please put your final answer within \boxed{{}}。render_prompt 只替换 {question}, 不会把 {{ 收成 {。模型看到的是双反斜杠和双大括号, 不是讲义里用来说明评分的 \boxed{{}}。评分器仍然在回答里找 \boxed{{...}}。

示例 (question_only, 格式对、答案错; 标准答案 3)。提示末尾是 \boxed{{}}, 回答写成了 \boxed{{7}}:

Prompt: ...How many bolts in total does it take? Please put your final answer within \boxed{{}}.

Response: ...Total bolts for all robe outfits: 5 + 2 = 7. ... \boxed{{7}}.

Ground truth: 3

format_reward: 1

answer_reward: 0

示例 (question_only, 格式错)。回答离开原题, 没有可解析的 boxed 答案:

Response: Write your answer below to the answer above
Ground truth: 160
format_reward: 0
answer_reward: 0

示例 (r1_zero, 格式对、答案错)。<answer> 里写的是每天 \$18, 标准答案就是 18, 但 answer_reward 仍是 0。评分没有把 \$18 收成 18:

Response: ... </think> <answer> Janet has 16 - 3 (for breakfast) - 4 (for her muffins) = 9 fresh duck eggs daily. She sells her fre
Ground truth: 18
format_reward: 1
answer_reward: 0

示例 (r1_zero, 格式错)。标签没有配成对, 数字也不是标准答案:

Response: ... </think> Since white fiber is half mode of the blue one ... So, it takes 1 + 3 + 2 = 6 bolts.</answer>
Ground truth: 3
format_reward: 0
answer_reward: 0

示例 (r1_zero_three_shot, 格式对、答案错)。标签完整, 标签里的数本身是错的:

Response: ... </think> <answer> \$120 </answer>
Ground truth: 18
format_reward: 1
answer_reward: 0

这几条能说明的是现象, 不是测试集上的 10 条比例: 类别 2 里可以出现「人看是对的、解析器判错」(上面的 \$18); three-shot 的类别 2 也可以只是算术错。类别 3 则是标签或 boxed 根本不完整。

(b)

只把题目放进 question_only 不够。模板虽然要求把答案放进 \boxed{{}}, 模型仍经常不答题。上面那条格式错的回答 boxed 答案, 而是写了“Write your answer below to the answer above”。

零样本 r1_zero 把大约一半输出拉进 </think> <answer>, 但推理常常复读题面或中途改口, 1319 题里格式和答案同时正确的只有 1 题。

三样本 r1_zero_three_shot 让模型模仿示例的标签。格式错误降到 74/1319 (5.6%), 答对升到 267/1319 (20.2%)。上面那条验证集回答把 \$120 放进完整的 <answer>, 标准答案是 18, 错在算术, 不在标签。

4.1 推导 on-policy GRPO

4.1.5 Baseline

题目 (baseline_calcs): 计算 policy-gradient estimator 的方差 (5 分) 题目来源: cs336_spring2026_assignment5_align

Compute the variance of the policy gradient estimator (5 points)

Let π_θ define a policy over the binary action space $\mathcal{A} = \{0, 1\}$ with $\pi_\theta(A = 1) = p = \sigma(\theta)$, where σ represents the sigmoid function $\sigma(\theta) = \frac{1}{1+e^{-\theta}}$. Our binary reward function will assign reward 1 to action $A = 1$ and reward 0 otherwise, or $r(A) = 1\{A = 1\}$.

(a) Let the policy gradient estimator be given by

$$\frac{1}{n} \sum_{i=1}^n r(A_i) \nabla_\theta \log \pi_\theta(A_i), \quad (18)$$

given n samples $A_i \stackrel{iid}{\sim} \pi_\theta$. What is the variance of this estimator?

Deliverable: An expression in terms of n and p , accompanied by a derivation.

(b) Let the baseline-adjusted policy gradient estimator be given by

$$\frac{1}{n} \sum_{i=1}^n (r(A_i) - b) \nabla_{\theta} \log \pi_{\theta}(A_i), \quad (19)$$

given n samples $A_i \stackrel{iid}{\sim} \pi_{\theta}$. What is the variance of this estimator?

Deliverable: An expression in terms of n , b , and p , accompanied by a derivation and some discussion.

(c) What is the resulting variance if we substitute the “population mean” baseline $b = p$? Compare this variance to that of the unadjusted policy gradient estimator: is it always lower, always higher, or sometimes higher or lower depending on p ?

答案

$\nabla_{\theta} \log \pi_{\theta}(A = 1) = 1 - p$, $\nabla_{\theta} \log \pi_{\theta}(A = 0) = -p$ 。因此 $A = 0$ 时 (18) 式的被加项为 0; $A = 1$ 时为 $1 - p$ 。单样本 Z 以概率 p 取值 $1 - p$, 否则为 0。

$$\mathbb{E}[Z] = p(1 - p), \quad \mathbb{E}[Z^2] = p(1 - p)^2,$$

$$\text{Var}(Z) = p(1 - p)^2 - p^2(1 - p)^2 = p(1 - p)^3.$$

n 个独立样本平均后, (a) 的方差是 $p(1 - p)^3/n$ 。

(b) $A = 1$ 时被加项为 $(1 - b)(1 - p)$, $A = 0$ 时为 bp 。期望仍是 $p(1 - p)$, 所以任意常数 b 都不改变这个期望。二阶矩是

$$\mathbb{E}[Z_b^2] = p(1 - b)^2(1 - p)^2 + (1 - p)b^2p^2.$$

减去 $[p(1 - p)]^2$ 后得到 $\text{Var}(Z_b) = p(1 - p)(1 - p - b)^2$ 。平均后的方差是

$$\frac{p(1 - p)(1 - p - b)^2}{n}.$$

$b = 0$ 时退回 (a)。 b 离 $1 - p$ 越近, 方差越小。

(c) 代入 $b = p$, 方差为 $p(1 - p)(1 - 2p)^2/n$ 。与 (a) 相比, 比值是 $(1 - 2p)^2/(1 - p)^2$ 。 $p < 2/3$ 时更低, $p = 2/3$ 时相等, $p > 2/3$ 时更高。 p 接近 1 时, $A = 0$ 很少出现, 但 $b = p$ 仍给这些样本加上量级为 p^2 的项, 噪声比不减 baseline 更大。

4.2 实现 on-policy GRPO

4.2.1 使用 Hugging Face 模型

题目 (tokenize_prompt_and_output) : Prompt 与 output tokenization (1 分) 题目来源: cs336_spring2026_assignment1

Prompt and output tokenization (1 point)

Deliverable: Implement a method `tokenize_prompt_and_output` that tokenizes the prompt and output separately, concatenates them together without inserting special tokens, and constructs a `response_mask`. The following interface is recommended: `def tokenize_prompt_and_output(`

`prompt_strs: list[str],`

`output_strs: list[str],`

`tokenizer: PreTrainedTokenizer,`

) -> dict[str, torch.Tensor]:

Tokenize the prompt and output strings, and construct a mask aligned with labels that is 1 for response tokens and 0 for other tokens (prompt or padding).

Args:

- prompt_strs: list[str] List of prompt strings.
- output_strs: list[str] List of output strings.
- tokenizer: PreTrainedTokenizer Tokenizer to use for tokenization.

Returns:

- dict[str, torch.Tensor]. Let prompt_and_output_lens be a list containing the lengths of the concatenated tokenized prompt and output strings. Then the returned dictionary should have the following keys:
 - input_ids torch.Tensor of shape (batch_size, max(prompt_and_output_lens) - 1): the tokenized prompt and output strings, with the final token sliced off.
 - labels torch.Tensor of shape (batch_size, max(prompt_and_output_lens) - 1): shifted input ids, i.e., the input ids without the first token.

14

- response_mask torch.Tensor of shape (batch_size, max(prompt_and_output_lens) - 1): a mask aligned with labels, with value 1 where the corresponding label token is part of the response and 0 otherwise.

To test your code, implement adapters.run_tokenize_prompt_and_output . Then, run the test with `uv run pytest -k test_tokenize_prompt_and_output` and make sure your implementation passes it.

答案

设 prompt_and_output_lens 为每个样本拼接后 token 序列长度的列表。返回 dictionary 应包含:

- input_ids: shape 为 (batch_size, max(prompt_and_output_lens)-1); 即拼接后的 token 序列去掉最后一个 token。
- labels: shape 相同; 即 shifted input IDs, 也就是去掉第一个 token。
- response_mask: shape 相同; 与 labels 对齐, response 对应的 label token 为 1, prompt 和 padding 位置为 0。

测试方法: 在 tests/adapters.py 中实现 adapters.run_tokenize_prompt_and_output, 然后运行:

`uv run pytest -k test_tokenize_prompt_and_output`

模型前向可以写成:

```
input_ids = train_batch["input_ids"].to(device)
labels = train_batch["labels"].to(device)
logits = model(input_ids).logits
```

下一步实现一个函数, 计算 response 中每个 token 在模型下的 conditional log-probability。这是计算 policy gradient 的核心 primitive。在 RL 中也常常记录 per-token entropy, 因此函数还需要支持 return_token_entropy。

实现在 cs336_alignment/grpo.py 的 tokenize_prompt_and_output。prompt 与 output 分开编码, 不加 special token, 直接拼接。input_ids 去掉最后一个 token, labels 去掉第一个 token, response_mask 与 labels 对齐。2026-09-29 本机 Mac 上 .venv/bin/python -m pytest tests/test_grpo.py 共 19 passed, 其中包含 test_tokenize_prompt_and_output。

题目 (get_response_log_probs) : Response 的 log-probability 与 entropy (1 分) 题目来源: cs336_spring2026_assignment

Response log-probs (and entropy) (1 point)

Deliverable: Implement a method get_response_log_probs that gets per-token conditional logprobabilities (given the previous tokens) from a causal language model, and optionally the entropy of the model's next-token distribution.

The following interface is recommended: `def get_response_log_probs(`

model: PreTrainedModel,

input_ids: torch.Tensor,

labels: torch.Tensor,

return_token_entropy: bool = False,

) -> dict[str, torch.Tensor]:

Args:

- model: PreTrainedModel HuggingFace model used for scoring (placed on the correct device and in inference mode if gradients should not be computed).
- input_ids: torch.Tensor shape (batch_size, sequence_length), concatenated prompt + response tokens as produced by your tokenization method.
- labels: torch.Tensor shape (batch_size, sequence_length), labels as produced by your tokenization method.
- return_token_entropy: bool If True, also return per-token entropy.

Returns:

- dict[str, torch.Tensor].
 - “log_probs” shape (batch_size, sequence_length), conditional log-probabilities $\log p_{\theta}(x_t | x_{<t})$.
 - “token_entropy” optional, shape (batch_size, sequence_length), per-token entropy for each position (present only if return_token_entropy=True).

To test your code, implement `adapters.run_get_response_log_probs`. Then run `uv run pytest`

`-k test_get_response_log_probs` and ensure the test passes.

答案

$$\log p_{\theta}(x_t | x_{<t}).$$

- “token_entropy”: 可选, shape 相同, 仅在 `return_token_entropy=True` 时出现。

测试:

`uv run pytest -k test_get_response_log_probs`

`get_response_log_probs` 对 `model(input_ids).logits` 做 `log_softmax`, 再按 `labels` 取出对数概率。`return_token_entropy=True` 时用同一分布计算每个位置的熵。同一轮测试中的 `test_get_response_log_probs` 通过。

4.2 实现 on-policy GRPO

4.2.3 GRPO 的组成部分

题目 (`compute_rollout_rewards`) : 计算 rollout reward (1 分) 题目来源: `cs336_spring2026_assignment5_alignment.p`

Computing the rewards of rollouts (1 point)

Deliverable: Implement a method `compute_rollout_rewards` that calculates raw rewards for each rollout response.

The following interface is recommended: `def compute_rollout_rewards(`

`reward_fn: Callable[[str, str], dict[str, float]],`

`rollout_responses: list[str],`

`repeated_ground_truths: list[str],`

`) -> tuple[torch.Tensor, dict[str, float]]:`

Compute rewards for a list of rollout responses, along with metadata for the reward components.

Args:

- `reward_fn`: Callable[[str, str], dict[str, float]] Scores the rollout responses against the ground truths, producing a dict with keys “reward”, “format_reward”, and “answer_reward”.
- `rollout_responses`: list[str] Rollouts from the policy. The length of this list is `rollout_batch_size = n_prompts_per_rollout_batch * group_size`.
- `repeated_ground_truths`: list[str] The ground truths for the examples. The length of this list is `rollout_batch_size`, because the ground truth for each example is repeated `group_size` times.

Returns:

- tuple[torch.Tensor, dict[str, float]].
 - `raw_rewards` shape (rollout_batch_size,). Unnormalized rewards for each rollout response.
 - `metadata` Reward statistics to log. At minimum, include the mean total and format rewards over the rollout batch.

To test your code, implement `adapters.run_compute_rollout_rewards`. Then, run the test with `uv run pytest -k compute_rollout_rewards` and make sure your implementation passes it.

答案

`compute_rollout_rewards` 对每条 rollout 调用 `reward_fn`, 把 “reward” 收成 float32 向量, 并记录平均 total、format、answer reward。test_compute_rollout_rewards 通过。

题目 (`compute_group_normalized_rewards_group`) : Group normalization (1 分) 题目来源: cs336_spring2026_assignment1

Group normalization (1 point)

Deliverable: Implement a method `compute_group_normalized_rewards` that normalizes raw rewards within their groups and returns the normalized rewards along with any metadata you think is useful.

For now, you only need to support `baseline = “mean”` and `advantage_normalizer = “std”`. Feel free to raise a `NotImplementedError` for unsupported inputs. In later parts of the assignment we will implement the other options and run ablations. Remember to add `advantage_eps` to the normalizer to avoid division by zero.

The following interface is recommended: `def compute_group_normalized_rewards(`

`raw_rewards: torch.Tensor,`

`group_size: int,`

`baseline: Literal[“mean”, “none”] = “mean”,`

`advantage_eps: float = 1e-6,`

`advantage_normalizer: Literal[“std”, “none”, “mean”] = “std”,`

`)`:

Compute advantages by applying the requested baseline and normalization within each group.

Args:

- `raw_rewards`: torch.Tensor shape (rollout_batch_size,). Unnormalized rewards for each rollout response, where `rollout_batch_size = n_prompts_per_rollout_batch * group_size`.
- `group_size`: int Number of responses per question (group).
- `baseline`: Literal[“mean”, “none”] For this problem, support mean, which subtracts the pergroup mean reward. Later, none will mean no baseline subtraction.
- `advantage_eps`: float Small constant to avoid division by zero in normalization.

- `advantage_normalizer`: Literal[“std”, “none”, “mean”] For this problem, support std, which divides by the per-group standard deviation. Later, none will mean no normalization and mean will mean divide by the per-group mean reward.

Returns:

- tuple[torch.Tensor, dict[str, float]].
 - `advantages` shape (rollout_batch_size,). Group-normalized rewards for each rollout response.
 - `metadata` your choice of other statistics to log (e.g. mean, std, max/min of rewards).

To test your code, implement `adapters.run_compute_group_normalized_rewards`. Then, run the test with `uv run pytest -k compute_group_normalized_rewards_grpo` and make sure your implementation passes it.

答案

$$J_{\theta}^{\text{GRPO-on-policy}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i} \log \pi_{\theta}(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}). \quad (32)$$

这个表达式并不是传统意义上“训练过程中应不断下降”的 loss，而只是一个求导后会得到我们想要的 policy gradient 的 objective。

由于 PyTorch optimizer 默认做 gradient descent，因此实现时应返回这个 objective 的负值。

`compute_policy_gradient_loss` 负责计算每个 token、每个 sequence 的 loss term；随后 `aggregate_loss_across_microbatch` 再在 token 和 sequence 维度上做 aggregation。

`compute_group_normalized_rewards` 在 `baseline=“mean”`、`advantage_normalizer=“std”` 时，按 `group_size` 分组，减去组内均值，再除以该组的样本标准差加 `advantage_eps`。`test_compute_group_normalized_rewards_grpo` 通过。

题目 (`compute_policy_gradient_loss_on_policy`) : On-policy policy gradient (1 分) 题目来源: cs336_spring2026_assignment3

On-policy policy gradient (1 point)

Deliverable: Implement a method `compute_policy_gradient_loss` that computes the per-token policy-gradient loss given raw rewards or pre-computed advantages.

For now, we will assume all rollouts are on-policy, so you only need to support `importance_reweighting_method = “none”`, and you can ignore the `old-log-prob` and `clipping` arguments. Feel free to raise a `NotImplementedError` for unsupported inputs. We will implement clipping later in the assignment.

The following interface is recommended: `def compute_policy_gradient_loss(`

`raw_rewards_or_advantages: torch.Tensor,`

`policy_log_probs: torch.Tensor,`

`importance_reweighting_method: Literal[“none”, “noclip”, “grpo”, “gspo”] = “none”,`

`old_log_probs: torch.Tensor | None = None,`

`cliprange: float | None = None,`

`response_mask: torch.Tensor | None = None,`

`) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:`

Compute the policy-gradient loss at every token, where `raw_rewards_or_advantages` is either the raw reward or an already-normalized advantage.

Args:

- `raw_rewards_or_advantages`: torch.Tensor Shape (batch_size,) or (batch_size, 1), scalar reward/advantage for each rollout response.

- `policy_log_probs`: torch.Tensor Shape (batch_size, sequence_length), logprobs for each token.
- `importance_reweighting_method`: Literal["none", "noclip", "grpo", "gspe"] "none": no importance reweighting; "noclip": apply importance reweighting without clipping; "grpo": do PPO/GRPO-style token-level reweighting and clipping; "gspe": do GSPO-style sequence-level reweighting and clipping.
- `old_log_probs`: torch.Tensor | None Required unless `importance_reweighting_method` = "none"; shape (batch_size, sequence_length).
- `cliprange`: float | None = None Clip parameter ϵ , required when `importance_reweighting_method` is "grpo" or "gspe".
- `response_mask`: torch.Tensor | None = None Optional shape (batch_size, sequence_length) mask over response tokens. Required for GSPO implementations that average the sequencelevel log-ratio over response tokens only.

Returns:

- tuple[torch.Tensor, dict[str, torch.Tensor]].
 - `per_token_policy_gradient_loss` Shape (batch_size, sequence_length), the per-token policy-gradient loss (to be aggregated across the batch and sequence dimensions in the training loop).
 - `metadata` Statistics from the underlying loss call, such as clip-fraction components.

To test your code, implement `adapters.run_compute_policy_gradient_loss`. Then run `uv run pytest -k test_compute_policy_gradient_loss_on_policy` and ensure the test passes.

答案

`importance_reweighting_method="none"` 时，逐 token loss 是 $-(\text{advantage} * \log_prob)$ 。取负号是因为优化器做梯度下降。通过。

题目 (`aggregate_loss_across_microbatch_sequence`) : 跨 token 与 sequence 聚合 loss (0.5 分) 题目来源: cs336_spring

Aggregate loss across tokens and sequences (0.5 points)

Deliverable: Implement a method `aggregate_loss_across_microbatch` that takes in the pertoken policy-gradient loss and the response mask, and outputs the average loss. For now, we will use standard GRPO aggregation, which involves first averaging within each sequence, and then averaging over sequences, so you can assume `loss_normalization = "sequence"`. Feel free to raise a `NotImplementedError` for unsupported inputs.

The following interface is recommended: `def aggregate_loss_across_microbatch(`

`per_token_policy_gradient_loss: torch.Tensor,`

`mask: torch.Tensor,`

`loss_normalization: Literal["sequence", "constant"] = "sequence",`

`normalization_constant: int | None = None,`

`) -> torch.Tensor:`

Aggregate the per-token policy-gradient loss according to the response mask and `lossnormalization` strategy.

Args:

- `per_token_policy_gradient_loss`: torch.Tensor Shape (batch_size, sequence_length), the per-token policy-gradient loss (to be aggregated across the batch and sequence dimensions in the training loop).
- `mask` torch.Tensor of shape (batch_size, sequence_length) denoting which positions should be included in the loss.
- `loss_normalization`: Literal["sequence", "constant"] = "sequence" "sequence": average loss over each sequence, then average over sequences; "constant": normalize total loss by a constant.

- `normalization_constant`: `int | None` = `None` The constant to divide total loss by; required if `loss_normalization = "constant"`.

Returns:

- `loss`: `torch.Tensor` A scalar containing the average loss. Make sure you can later call backward on this loss.

To test your code, implement `adapters.run_aggregate_loss_across_microbatch` . Then run `uv run pytest -k test_aggregate_loss_across_microbatch_sequence` and ensure the test passes.

20

答案

`loss_normalization="sequence"` 时，先对每条序列里 `mask` 为真的位置取平均，再对序列取平均。`test_aggregate_loss_across_microbatch_sequence` 通过。

4.2 实现 on-policy GRPO

4.2.4 GRPO training step

题目 (`grpo_train_step_standard_on_policy`) : GRPO train step (5 分) 题目来源: `cs336_spring2026_assignment5_align`

GRPO train step (5 points)

Deliverable: Implement a single batch update for policy gradients, given the model, tokenizer, and rollouts. For this part of the assignment, you only need to implement this function for standard GRPO in the on-policy setting, or `baseline = "mean"`, `advantage_normalizer = "std"`, `importance_reweighting_method = "none"`, and `loss_normalization = "sequence"`. Feel free to raise a `NotImplementedError` for unsupported inputs.

The following interface is recommended: `def grpo_train_step(`

`model: PreTrainedModel,`

`tokenizer: PreTrainedTokenizer,`

`optimizer: Optimizer,`

`gradient_accumulation_steps: int,`

`max_grad_norm: float | None,`

`reward_fn: Callable[[str, str], dict[str, float]],`

`repeated_prompts: list[str],`

`rollout_responses: list[str],`

`repeated_ground_truths: list[str],`

`group_size: int,`

`# Reward normalization`

`baseline: Literal["mean", "none"] = "mean",`

`advantage_eps: float = 1e-6,`

`advantage_normalizer: Literal["std", "none", "mean"] = "std",`

`# Importance reweighting and clipping`

`importance_reweighting_method: Literal["none", "noclip", "grpo", "gspe"] = "none",`

old_log_probs: torch.Tensor | None = None,

cliprange: float | None = None,

Loss normalization

loss_normalization: Literal["sequence", "constant"] = "sequence",

normalization_constant: int | None = None,

) -> tuple[torch.Tensor, dict[str, torch.Tensor | float]]:

Execute forward-and-backward passes, with gradient_accumulation_steps microbatches.

Args:

- model: PreTrainedModel HuggingFace model to train.
- tokenizer: PreTrainedTokenizer Tokenizer to use for tokenization.
- optimizer: Optimizer Optimizer for the model.
- gradient_accumulation_steps: int Number of microbatches per optimizer step.
- max_grad_norm: float | None If not None, clip the gradient norm to this value before calling optimizer.step().
- reward_fn: Callable[[str, str], dict[str, float]] Scores the rollout responses against the ground truths, producing a dict with keys "reward", "format_reward", and "answer_reward".
- repeated_prompts: list[str] The prompts for the examples. The length of this list is rollout_batch_size, because the prompt for each example is repeated group_size times.
- rollout_responses: list[str] Rollouts from the policy. The length of this list is rollout_batch_size = n_prompts_per_rollout_batch * group_size.
- repeated_ground_truths: list[str] The ground truths for the examples. The length of this list is rollout_batch_size, because the ground truth for each example is repeated group_size times.
- group_size: int Number of responses per question (group).
- baseline: Literal["mean", "none"] If mean, subtract the per-group mean reward; if none, do nothing.
- advantage_eps: float Small constant to avoid division by zero in normalization.
- advantage_normalizer: Literal["std", "none", "mean"] If std, divide by the per-group standard deviation; if none, do nothing; if mean, divide by the per-group mean reward.

• importance_reweighting_method: Literal["none", "noclip", "grpo", "gspo"] "none": no importance reweighting; "noclip": apply importance reweighting without clipping; "grpo": do

PPO/GRPO-style token-level reweighting and clipping; "gspo": do GSPO-style sequence-level reweighting and clipping.

• old_log_probs: torch.Tensor | None Required unless importance_reweighting_method =

"none"; shape (batch_size, sequence_length).

• cliprange: float | None = None Clip parameter ϵ , required when importance_reweighting_method is "grpo" or "gspo".

• loss_normalization: Literal["sequence", "constant"] = "sequence" "sequence": average loss over each sequence, then average over sequences; "constant": normalize total loss by a constant

(fixed for all of training).

• normalization_constant: int | None = None The constant to divide total loss by; required if loss_normalization = "constant".

Returns:

• tuple[torch.Tensor, dict[str, torch.Tensor]].

- loss scalar tensor. The batch loss, adjusted for gradient accumulation. We return this so we can log it.

- metadata Dict with metadata from the underlying loss call, gradient norm before clipping, and any other statistics you might want to log.

To test your code, implement `adapters.run_grpo_train_step`. Then run `uv run pytest -k test_grpo_train_step_standard_on_policy` and confirm it passes.

答案

`grpo_train_step` 先对整批算 reward 和 advantage，再按 microbatch 前向、反传。sequence normalization 下，每个 microbatch 的 loss 乘 `microbatch_size / batch_size`。全部反传之后用 `clip_grad_norm` 得到 clip 前的全局梯度范数，再 `optimizer.step()`，并把梯度清成 None。返回的 metadata 含 loss、grad_norm、token_entropy、mean_policy 时底层 `compute_policy_gradient_loss` 的 metadata 为空）。`test_grpo_train_step_standard_on_policy` 通过。完整 GSM8K 训练不在本题的单元测试里。

4.3 实验

题目 (`grpo_experiments_standard_on_policy`)：使用 GRPO 提升 OLMo-2-0425-1B 在 GSM8K 上的性能（约 2 B200 小时，10 分） 题目来源：cs336_spring2026_assignment5_alignment.pdf

Use GRPO to improve OLMo-2-0425-1B performance on GSM8K (2 B200 hrs) (10 points)

- Write a script that runs the GRPO training loop given a model name, prompt, training set file path, validation set file path, sampling hyperparameters, and training hyperparameters.

At a high level, the script will involve first initializing the vLLM server, wandb logging, datasets, model, and optimizer. Then, the training loop involves repeating the following: sync weights with the vLLM server, produce training rollouts, take policy gradients on training batches, and periodically check performance on the validation set (while logging generations).

Deliverable: A script to run GRPO (standard, on-policy) on task GSM8K and model OLMo-2-0425-1B.

- Run the script for roughly 50 steps and confirm that you see validation rewards improving, along with sensible rollouts over time. Note: it is normal for the reward to start close to zero, and depending on the random seed it may take a few steps before we sample rollouts with non-zero rewards and the model starts improving.

Deliverable: Evidence that convinced you that your script is correct (e.g. validation rewards improving each step, reasonable-looking rollouts).

- Once you have confidence that the script is correct, run it using the hyperparameters provided above, with 4 random seeds, for OLMo-2-0425-1B on GSM8K. You should use the zero-shot r1_zero prompt. If the code is correct, you should see that validation rewards improve as the model is trained.

Log the following metrics, and produce plots of them over time, while noting the variance between runs (e.g. by plotting mean/std or min/max over runs).

- The loss.
- Gradient norm.
- Token entropy.
- Train rewards (total, format).
- Val rewards (total, format).
- Val average response length.
- Anything else you think could be useful for debugging.

Please also log the rollouts periodically and observe them. Do the responses improve as a result of training?

Deliverable: A few sentences of commentary and a plot for each metric, describing how it changes over training and how much variance there is between runs.

Deliverable: A few examples of the model rollouts before and after training.

Deliverable: A procedure that achieves a final validation accuracy of at least 25%, averaged across random seeds.

答案

2026-10-02, AutoDL 2× RTX PRO 6000 Blackwell (每张 96GB)。训练在物理 GPU 0, vLLM 在物理 GPU 1。模型是本地的 /root/autodl-tmp/models/OLMo-2-0425-1B。入口是 scripts/train_grpo.py, 四个 seed 由 scripts/run_all_experiments.sh standard 串行启动。Prompt 用 zero-shot r1_zero。超参按讲义: 6400 条训练、1024 条验证、学习率 1×10^{-5} 、每步 256 条 response、group size 8、32 次梯度累积、AdamW (β_1, β_2) = (0.9, 0.95)、weight decay 0、梯度裁剪 1.0、temperature 1.0、top-p 1.0、最长 512 token。每 10 步验证, 每 40 步保存训练 rollout。r1_zero 的总 reward 与 answer reward 一致, 所以验证总奖励曲线和答案准确率是同一回事。

- (a) 脚本按 model、prompt、训练/验证文件和采样、训练超参启动。循环里先把当前 policy 同步到 vLLM 并清空 prefix cache, 再采样 rollout, 做一次 on-policy 更新, 并按周期在验证集上记录生成。
- (b) 先用 seed 0 跑了 50 步, 目录 results/smoke_seed0。验证答案奖励在 step -1、20 仍是 0.1%, step 30 为 0.3%, step 40 升到 11.6%, step 50 为 29.0%。前 20 多步训练奖励和 loss 为 0: 一组 8 条回答奖励相同, advantage 为 0, 这一步没有参数更新。出现第一批答对的 rollout 之后, 验证准确率才上升。训练后的回答会把算式写进 <answer>, 不再只吐未闭合的推理段。
- (c) 四个 seed 各 200 步, 目录 results/standard_seed_0 到 standard_seed_3。最终验证答案奖励为 47.9%、45.6%、12.3%、46.7%, 平均 38.1%, 超过 25%。seed 2 把平均值拉低; 其余三个在 45% 以上, step 100 之后都停在大约 45%。seed 之间的差距主要出现在 step 40-80: seed 0 和 seed 3 先抬起来, seed 1 大约晚 20 步, seed 2 一直停在 12% 附近。

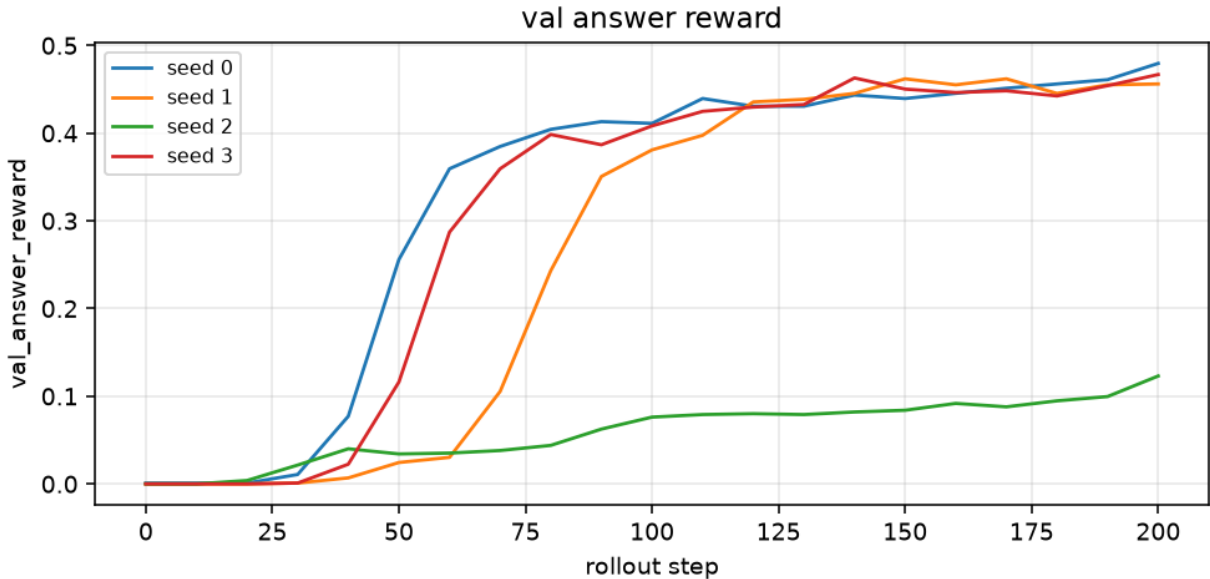


Figure 1: 验证答案奖励

验证格式奖励四个 seed 都升到 84%-97%。总奖励与答案奖励重合。格式对、答案错的比例在成功的 seed 上下降, 在 seed 2 上仍然很高: 它的格式奖励是 97.1%, 答案奖励只有 12.3%。

训练总奖励和验证答案奖励同步。成功的 seed 后半段训练奖励大约 0.45-0.55; seed 2 多停在 0.15 以下。格式奖励同样上升, seed 之间的差别比总奖励小。

loss 是取负后的 policy gradient, 所以答对的 token 变多时它降到 0 以下, 而不是朝着 0 做交叉熵。前 20-30 步四个 seed 的 loss 和梯度范数都是 0。之后 loss 落到大约 -0.04 到 -0.01, 没有发散。这里记录的 grad_norm 是裁剪前的范数, 裁剪上限是 1.0。seed 0、1、3 的中位数是 0.71-0.84, 200 步里超过 1 的只有 5、12、20 步。seed 2 高一档: 143/200 步大于 1, 中位数 1.48, 最大 2.81, 这些步的更新被裁到 1.0。

Token entropy 开头在 1 附近波动, step 40 之后随准确率升降到 0.15-0.2。seed 2 降得更晚, 后半段停在大约 0.3, 比另外三个更不确定。这张图把该步全部回答 token 都算进平均, 包括整组优势为 0、因此没有参数更新的步。grpo_train_step 只对送进模型的非零优势序列计算 entropy, 后面的实验若用这份代码, 熵的数值不能和这张图直接比。

成功的三个 seed 在准确率上升前会先把回答缩短到大约 50-80 token, 然后再拉长到 130-160。seed 2 从大约 100 token 掉到 15 左右, 结束时只有 30.3。它学会了短的 <answer> 格式, 但经常把推理压成一行, 并算错。re 的 256 条训练回答里, 有 168 条仍含 prompt 里的占位句 “reasoning process here”, 例如 reasoning process here </think> <answer> 95 </answer>。

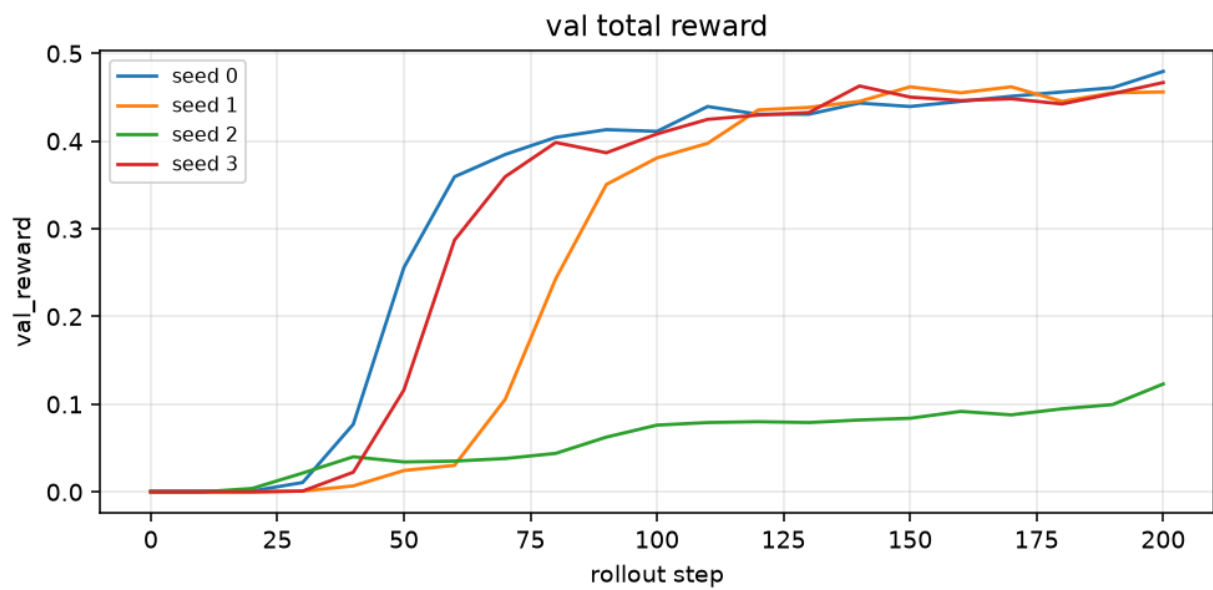


Figure 2: 验证总奖励

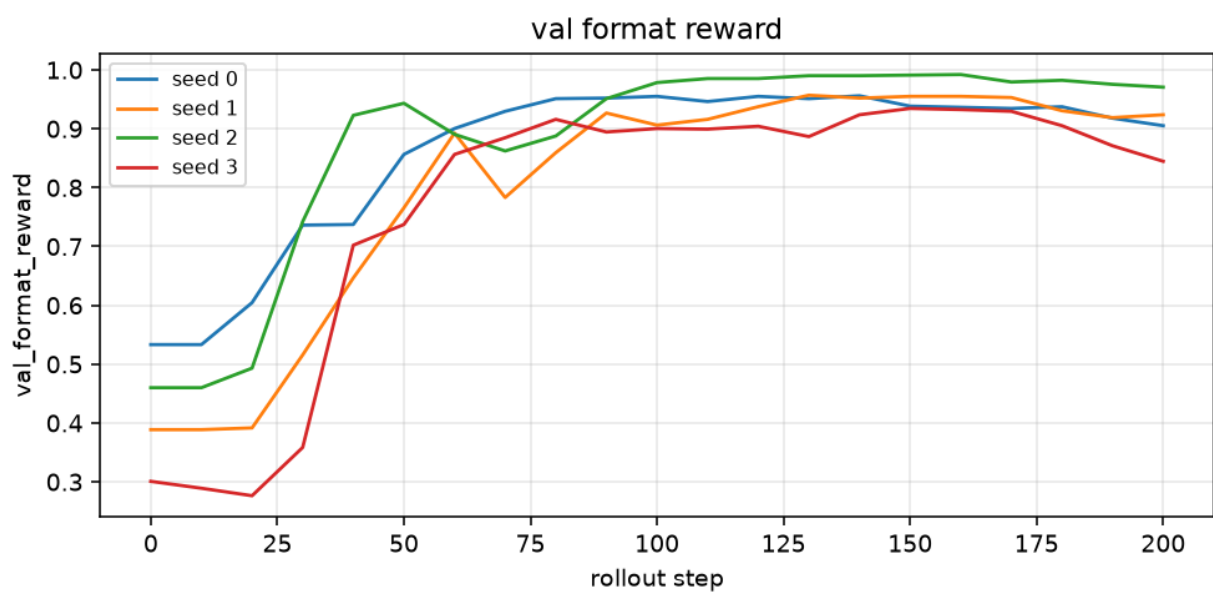


Figure 3: 验证格式奖励

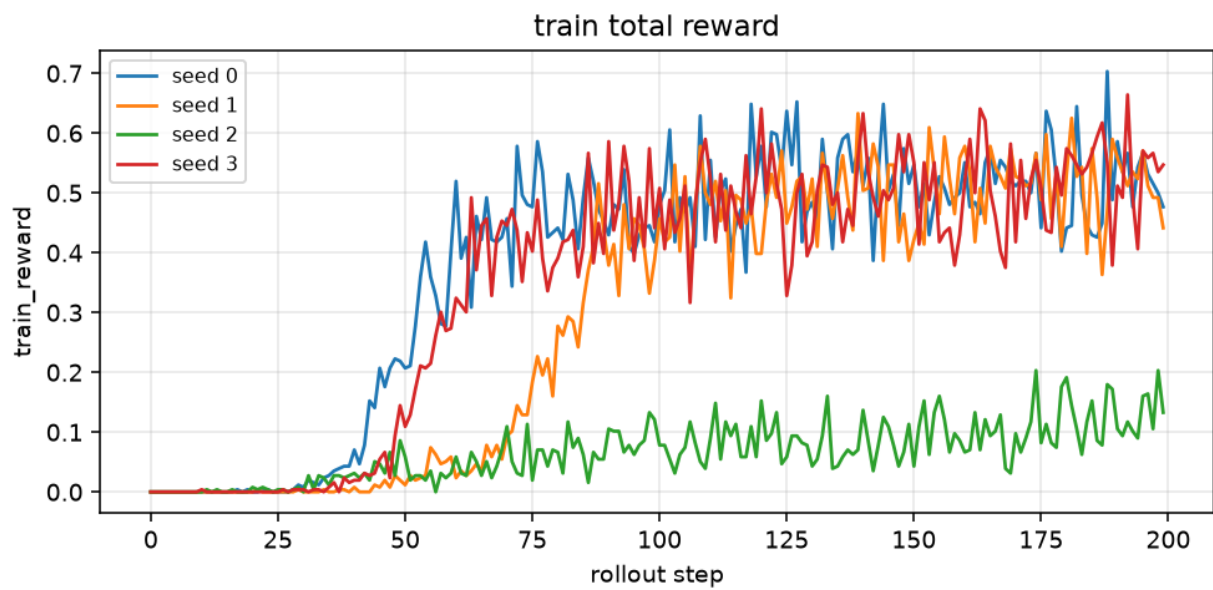


Figure 4: 训练总奖励

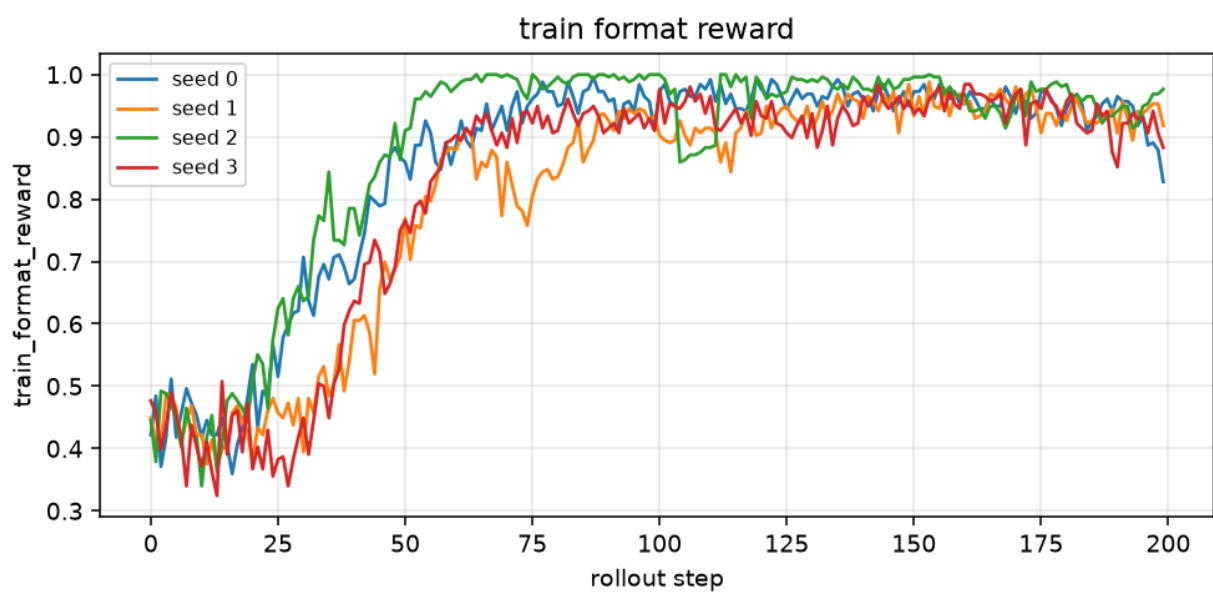


Figure 5: 训练格式奖励

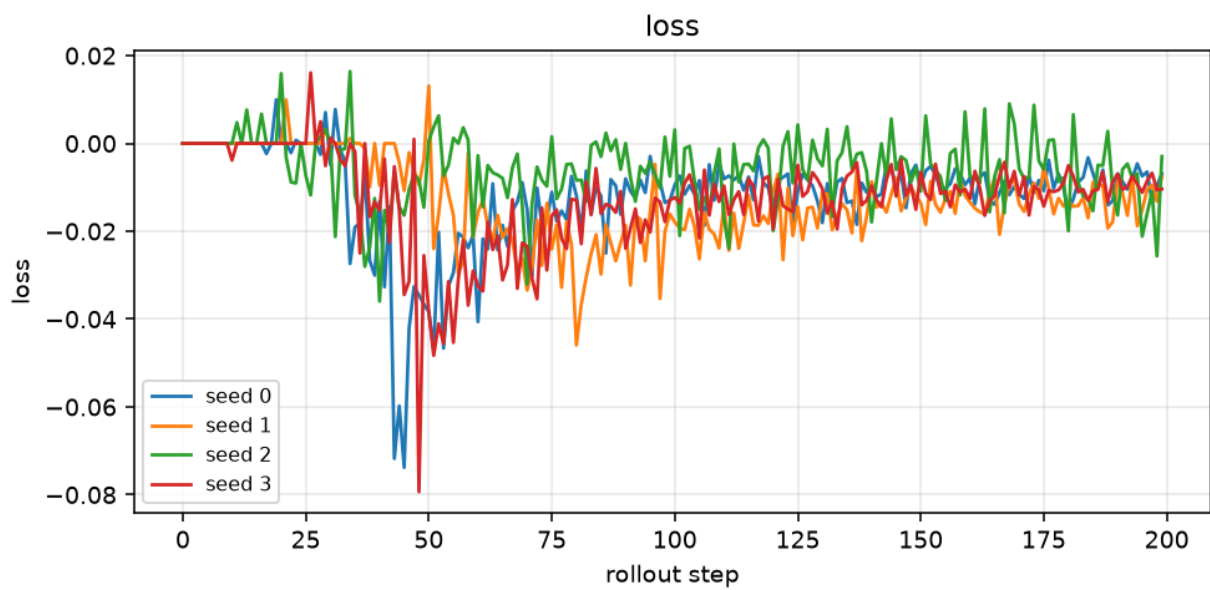


Figure 6: loss

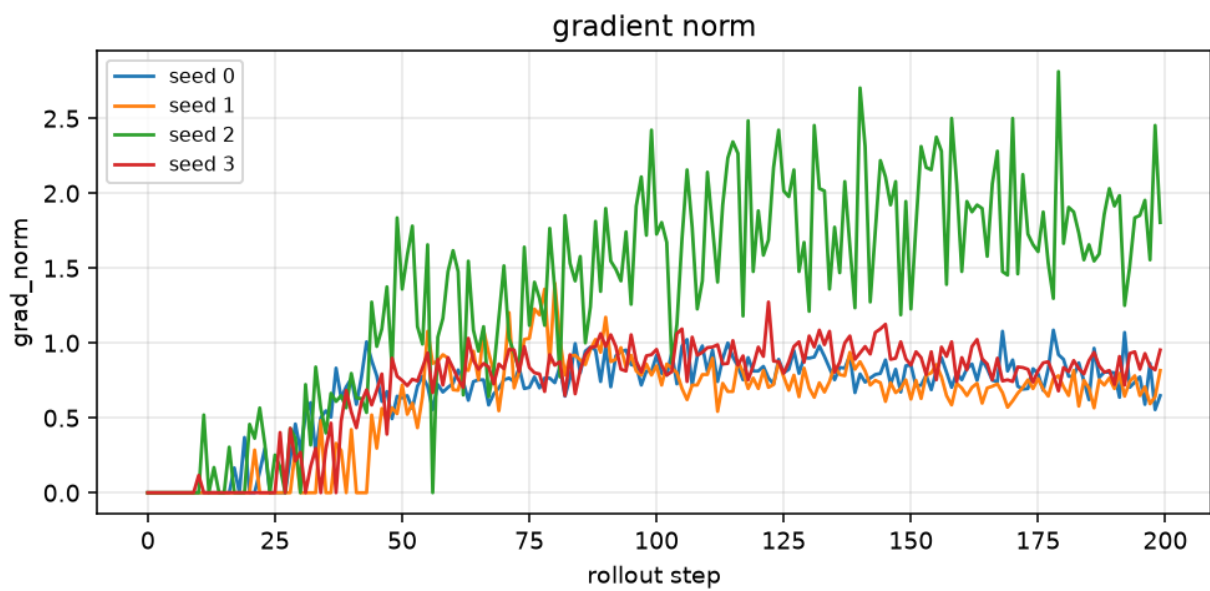


Figure 7: 梯度范数

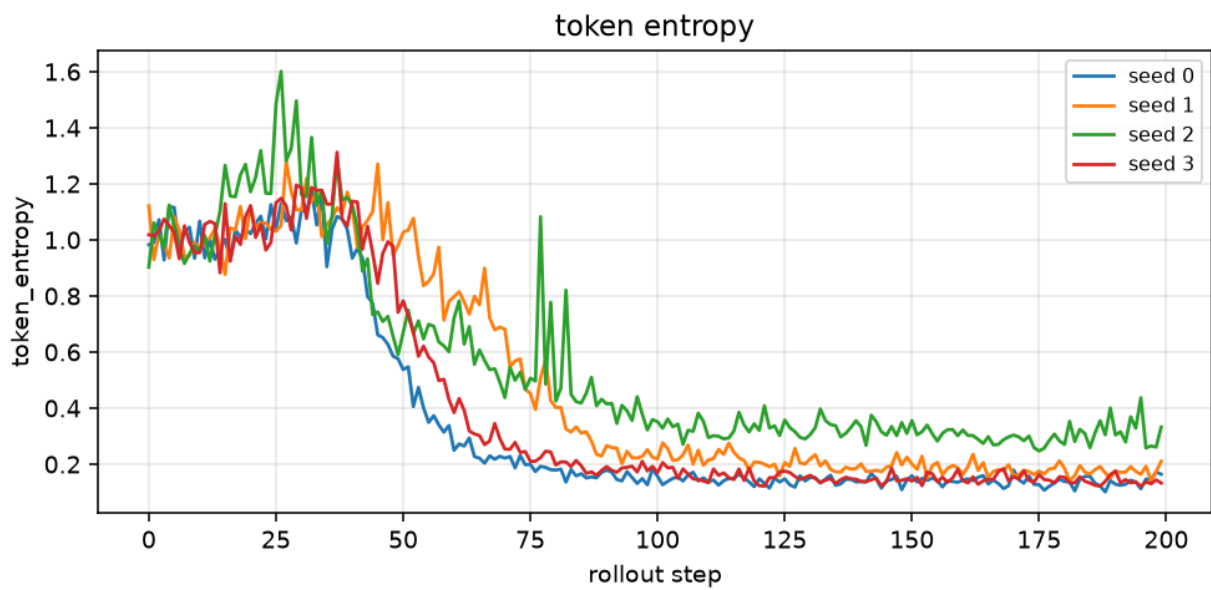


Figure 8: token entropy

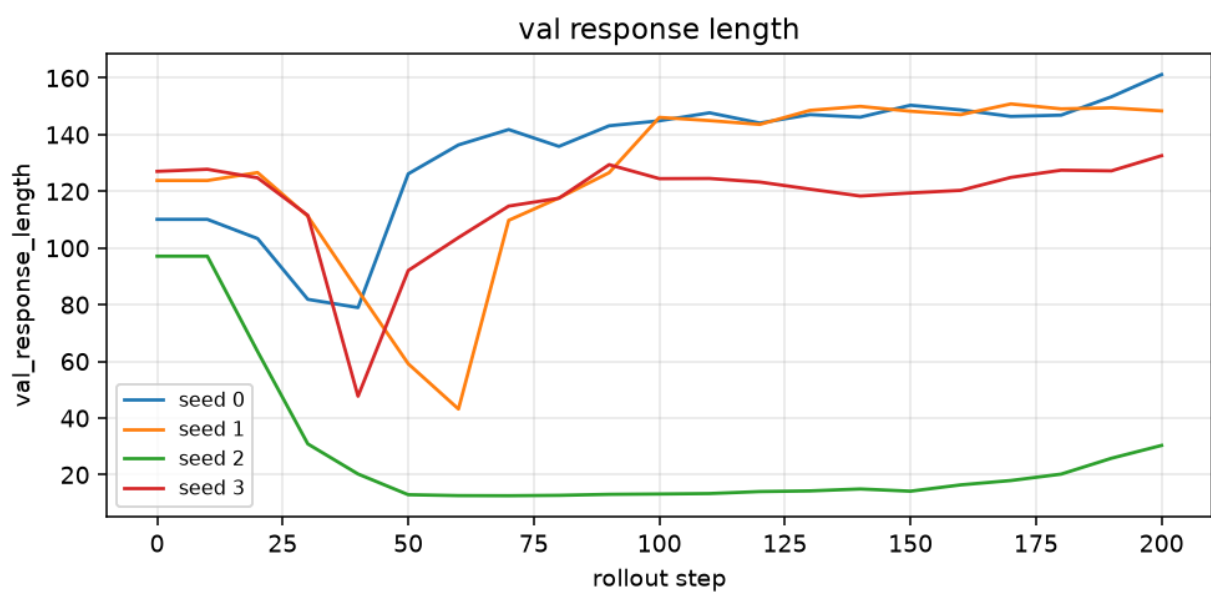


Figure 9: 验证回答长度

训练前 (seed 0, 验证, 格式错、答案错)。标准答案是 3:

A robe takes 2 bolts of blue fiber and half that much white fiber. How many bolts in total does it take?

It would take 1 bolt for each 50% of the white fiber if each piece takes the same amount of bolt. Since white fiber is half mode of the blue one, it would need 3 bolts for 50% of it. And for the actual 100%, which is 2 bolts, it would need 2 bolts. So, it takes $1 + 3 + 2 = 6$ bolts.

同一题训练后:

A robe takes 2 bolts of blue fiber and half that much white fiber. So, it takes 2 bolts of blue fiber and 1 bolt of white fiber. In total, it takes $2 + 1 = 3$ bolts of fiber for the robe. </think> <answer> 3 </answer>

另一题训练前把每周距离算成 810, 标签也没闭合。标准答案是 540。训练后写成 $60 \times 3 \times 3 = 540$, 并放进 <answer>。回答会随训练变对, 但 seed 2 说明这不是每个随机种子都会发生。

题目 (grpo_learning_rate): 调节 learning rate (约 4 B200 小时, 3 分) 题目来源: cs336_spring2026_assignment5_al

Tune the learning rate (4 B200 hrs) (3 points)

Starting from the suggested hyperparameters, perform a sweep over the learning rates (at least one learning rate less than the recommended default, and one learning rate greater than it), and report the final validation rewards (or note divergence if the optimizer diverges). Based on the amount of variance you observed in the previous part, you should decide how many random seeds you want to use for each training run. For the rest of the assignment, feel free to use your tuned learning rate instead of the suggested default, if you would like.

Deliverable: A plot of final validation reward versus learning rate, with a few sentences of commentary.

答案

4.3 里有一个 seed 停在 12.3%, 样本标准差大约 17 个百分点, 所以每个学习率仍用 4 个 seed。其余超参与标准 GRPO 相同。比较 5×10^{-6} 、讲义默认的 1×10^{-5} 和 3×10^{-5} 。 1×10^{-5} 直接用 results/standard_seed_*, 没有重跑。没有 loss 发散。

学习率	四个 seed 的最终验证答案奖励	平均	样本标准差
5×10^{-6}	39.4%, 39.5%, 36.5%, 38.8%	38.5%	1.4 pp
1×10^{-5}	47.9%, 45.6%, 12.3%, 46.7%	38.1%	17.2 pp
3×10^{-5}	53.2%, 4.1%, 4.7%, 49.6%	27.9%	27.2 pp

三个学习率的成功 seed 都落在 36%–53%。平均值几乎由失败 seed 决定: 5×10^{-6} 四次都成功, 平均和默认学习率一样 10^{-5} 的最好 seed 最高, 到 53.2%, 但有两次掉到 4% 左右, 最终回答只有约 11 token, 格式奖励却是 100%。学习率加大以后, 模型更容易收成极短的格式正确、答案错误的输出。后面的 prompt ablation 仍用 1×10^{-5} , 这样和上面的 r1_zero 结果可比。若只求稳定, 5×10^{-6} 更合适。

题目 (grpo_prompt_ablation): Prompt ablation (约 4 B200 小时, 3 分) 题目来源: cs336_spring2026_assignment5_a

Prompt ablation (4 B200 hrs) (3 points)

Run the GRPO script with the question_only prompt and the r1_zero_three_shot prompt, with a few random seeds. Compared to the zero-shot r1_zero prompt results above, how do these prompts perform? Which prompts have the best average reward, and the lowest variance? Are there other systematic differences in the other logged metrics? Based on the variance between runs, how confident are you in your findings?

Deliverable: Commentary, supported by plots for referenced metrics.

答案

学习率固定为 1×10^{-5} , 其余超参与 4.3 相同。question_only 和 r1_zero_three_shot 各跑 4 个 seed。zero-shot r1_zero 用上面的四个标准 run。

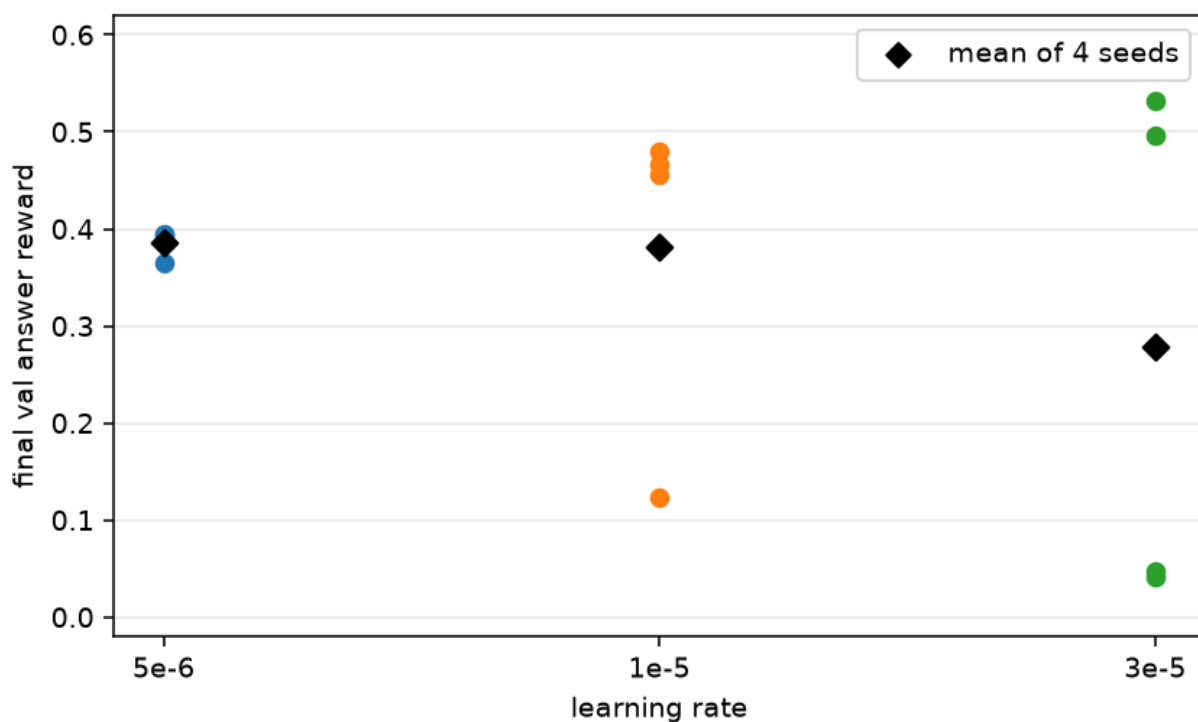


Figure 10: 最终验证答案奖励与学习率

prompt	四个 seed 的最终验证答案奖励	平均	样本标准差
r1_zero	47.9%, 45.6%, 12.3%, 46.7%	38.1%	17.2 pp
question_only	45.2%, 40.1%, 44.5%, 45.5%	43.8%	2.5 pp
r1_zero_three_shot	51.0%, 54.0%, 50.9%, 51.2%	51.8%	1.5 pp

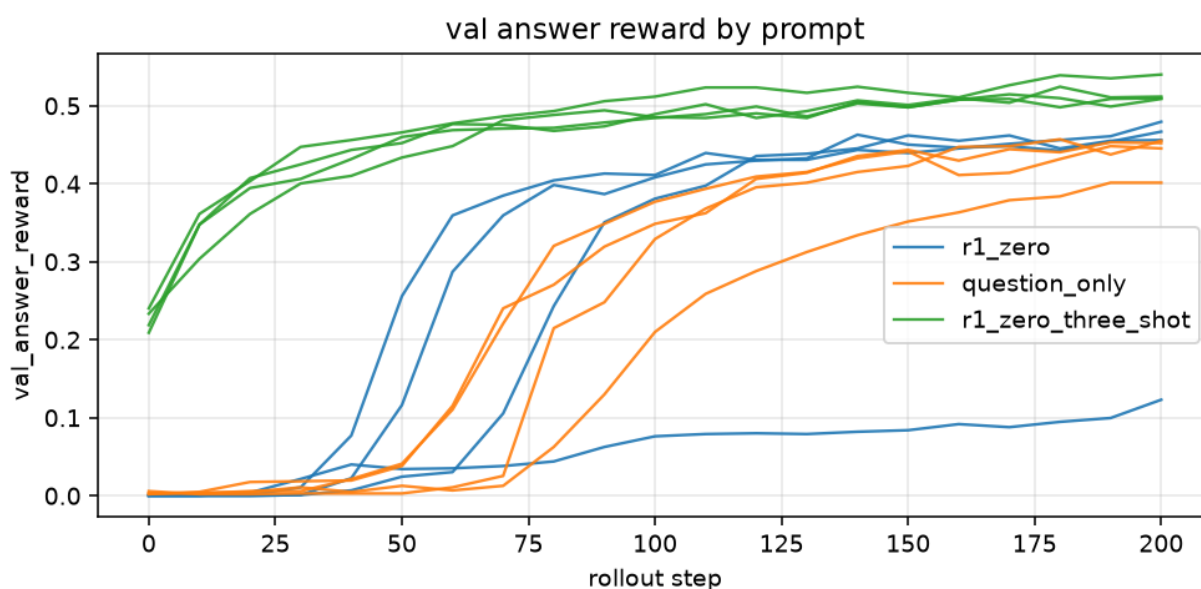


Figure 11: 不同 prompt 的验证答案奖励

平均奖励最高、波动最小的是 r1_zero_three_shot。四个 seed 都在 50.9%–54.0%，没有失败的 run。question_only 也是四次都成功，平均比 zero-shot r1_zero 高大约 6 个百分点，主要因为 r1_zero 有一次掉到 12.3%。去掉那一次，r1_zero 的另外三个 seed 和 question_only 重叠。

r1_zero_three_shot 在 step 0 的验证答案奖励就已经大约 20%，和第三章 few-shot 基线一致，然后继续升到 51%。另外两种 prompt 要从接近 0 开始学格式，step 40 以后才明显上升。

val format reward by prompt

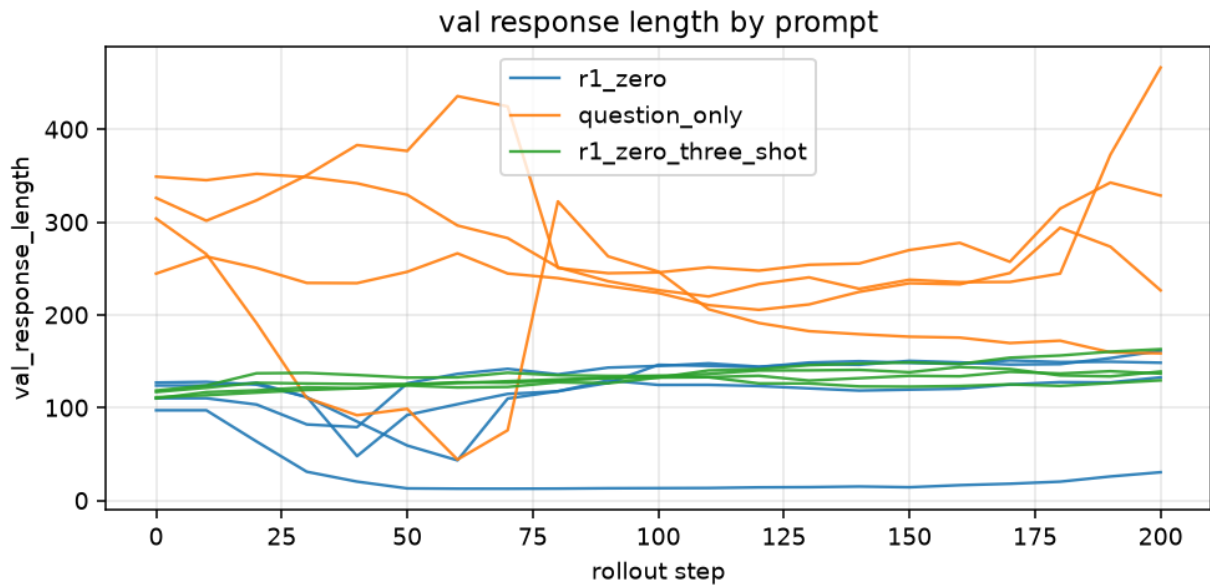


Figure 13: 不同 prompt 的验证回答长度

格式奖励最后都很高。差别在长度：r1_zero_three_shot 稳定在大约 130-160 token；成功的 r1_zero 也在这个范围，失败的那次缩到 30 token。question_only 没有这种短回答崩溃，但回答更长，四个 seed 的最终长度从 158 到 466 token，seed 之间差得很大。

四个 seed 里 r1_zero_three_shot 每一次都高于另外两种 prompt 的平均，组内标准差只有 1.5 个百分点，所以它最好这一是否稳定地优于 zero-shot r1_zero 就没那么确定：差距主要来自 r1_zero 的一个失败 seed。

5 RL 算法变体

5.1 Dr. GRPO

题目 (think_about_length_normalization)：思考 length normalization (1 分) 题目来源：cs336_spring2026_assignment1

Think about length normalization (1 point)

Before running any experiments, think about the difference between normalizing each sequence by sequence length, versus normalizing all sequences by the same constant. What are the pros and cons of each approach? Are there specific settings or examples where one approach seems better?

Deliverable: A few sentences of discussion.

答案

按每条序列自己的长度平均时，这条序列的总梯度是 advantage 乘上各 token score-gradient 的平均。advantage 只决定这条序列相对其他序列的系数，梯度和方向仍由 $\nabla_{\theta} \log \pi_{\theta}(y_t | x, y_{<t})$ 的大小和方向决定。序列长短不改变这 advantage 系数，所以短回答不会因为 token 少就被长回答淹没。从目标函数看，这等价于在 surrogate 里对每个 token 乘 $1/\text{len}(y)$ ，优化的不再是「每个 rollout 等权」的 $\nabla \mathbb{E}[r]$ ，而是带长度权重的 policy gradient。代价是长回答里每个 token 的梯度被缩小，模型缺少把正确推理写长的激励。用固定常数 Z 时，每个 token 的权重相同，长的正确回答贡献更多 token，总梯度更大，训练中回答容易变长。GSM8K 这种短答案任务里，如果目标是避免模型靠拉长输出刷更新，按序列长度归一化更合适；如果希望正确的长推理链 token 数相称的更新，固定常数更合适。

题目 (compute_group_normalized_rewards_drgrp)：Dr. GRPO group normalization (0.5 分) 题目来源：cs336_spring2026_assignment1

Problem (compute_group_normalized_rewards_drgrp): Dr. GRPO Group normalization (0.5 points)

Deliverable: Update your method compute_group_normalized_rewards to support advantage_normalizer = "none". While Dr. GRPO uses baseline = "mean", for this problem, please also add support for baseline = "none".

To test your code, implement `adapters.run_compute_group_normalized_rewards`. Then, run the test with:

```
uv run pytest -k compute_group_normalized_rewards_drgrp
```

and make sure your implementation passes it.

答案

`advantage_normalizer="none"` 时不再除以标准差。`baseline="mean"` 只减组内均值；`baseline="none"` 直接使用原始 reward。`test_compute_group_normalized_rewards_drgrp` 通过。

题目 (`aggregate_loss_across_microbatch_constant`) : Dr. GRPO loss aggregation (0.5 分) 题目来源: cs336_spring2026.

Dr. GRPO loss aggregation (0.5 points)

Deliverable: Update your method `aggregate_loss_across_microbatch` to support `loss_normalization = "constant"`.

To test your code, implement `adapters.run_aggregate_loss_across_microbatch`. Then run `uv run pytest -k test_aggregate_loss_across_microbatch_constant` and ensure the test passes.

答案

`loss_normalization="constant"` 时, `mask` 内的 loss 求和后除以给定常数。`test_aggregate_loss_across_microbatch_constant` 通过。训练步里这种归一化不再乘 `microbatch_size / batch_size`。

5 RL 算法变体

5.2 Rejection Fine-Tuning

题目 (`think_about_rft`) : 思考 RFT (2 分) 题目来源: cs336_spring2026_assignment5_alignment.pdf

Think about RFT (2 points)

Recall that the RFT objective (with constant normalization) is given by

$$J_{\theta} = \frac{1}{Z} \sum_x \sum_{j=1}^G 1\{r(y^{(j)} | x) = 1\} \log \pi_{\theta}(y^{(j)} | x), \quad (35)$$

where x is the prompt, each $y^{(j)}$ denotes a response $y^{(j)} \stackrel{iid}{\sim} \pi_{\theta}(\cdot | x)$, G is the number of generations we sample from the policy, Z is our constant normalizer, and r is the reward function. The RFT gradient is then given by $\nabla_{\theta} J_{\theta}$.

In contrast, the on-policy Dr. GRPO policy gradient estimator (i.e. GRPO with constant normalization and no std normalization) is given by

$$\frac{1}{Z} \sum_x \sum_{j=1}^G (r(y^{(j)} | x) - \mu) \nabla_{\theta} \log \pi_{\theta}(y^{(j)} | x), \quad (36)$$

where μ denotes the group mean $\mu = \frac{1}{G} \sum_{j=1}^G r(y^{(j)} | x)$. Assuming our reward function r is binary, compare and contrast these objectives. Do they have the same expectation? Which do you expect to have lower variance? Intuitively, are there situations where you would prefer one or the other?

Deliverable: A few sentences of discussion.

答案

reward 为 0/1 时, $1\{r = 1\} = r$ 。RFT 的期望梯度是 $\mathbb{E}[r \nabla_{\theta} \log \pi_{\theta}] = \nabla \mathbb{E}[r]$ 。Dr. GRPO 对每个样本用 $(r - \mu)$, μ 为组内均值。减去与 θ 无关的常数 baseline 不改变期望, 故 $G \rightarrow \infty$ 、 $\mu \rightarrow \eta(x) = \mathbb{E}[r | x]$ 时两者期望都对准 $\nabla \mathbb{E}[r]$ 。有限 G 时 μ 是组内随机变量, 梯度期望相对 $\nabla \mathbb{E}[r]$ 大约缩小因子 $(1 - 1/G)$ (例如 $G = 8$ 时约 12.5%), 只是近似无偏。

方差：4.1.5 的 $p < 2/3$ 只属于那个单参数、二元动作模型。那里 $\nabla_{\theta} \log \pi(A = 1) = 1 - p$ 、 $\nabla_{\theta} \log \pi(A = 0) = -p$ 由动作唯一确定，所以 $b = 0$ 时单样本方差是 $p(1 - p)^3$ ， $b = p$ 时是 $p(1 - p)(1 - 2p)^2$ 。二元 reward 只让 RFT 的乘数变成 $r \in \{0, 1\}$ 、Dr. GRPO 的乘数变成 $r - \mu$ 。语言模型的 $\nabla_{\theta} \log \pi(y | x)$ 是各 token score 之和，知道对错之后它仍随回答内容和长度变化，并不是 $1 - p$ 或 $-p$ 。因此不能用 $p < 2/3$ 预言 GSM8K 上谁的方差更低。

在 score-gradient 的尺度在组内差不多的条件下，我预期 Dr. GRPO 的方差更低：全对的组里 RFT 仍加上这些 $\nabla \log \pi$ ，Dr. GRPO 的 advantage 是 0；混合组里 $r - \mu$ 的幅度也小于 RFT 的 0/1 权重。如果正确回答系统性更长、score 范数更大，这个 centering 就不一定再降低梯度方差，那时二元 reward 本身决定不了谁更低。

行为差异：baseline="mean" 时全组同 reward（全对或全错）advantage 全为 0，该组无梯度；RFT (baseline="none") 的权重是 0，Dr. GRPO 减均值后也是 0。只想加权模仿正确 rollout、且不想全对组被 baseline 抹平时，更倾向 RFT。

5 RL 算法变体

5.3 MaxRL

题目 (derive_difficulty_reweightings)：推导不同 advantage normalizer 所引入的 difficulty reweighting (6 分) 题目来源：cs336_spring2026_assignment5_alignment.pdf

Derive difficulty reweightings induced by different advantage normalizers (6 points)

Recall that in aggregate over prompts, the standard policy gradient is given by

$$\nabla_{\theta} J_{\theta} = \nabla_{\theta} \mathbb{E}_{x \sim \rho} \left[\mathbb{E}_{y \sim \pi_{\theta}} r(y | x) \right]. \quad (39)$$

where ρ is our prompt distribution over prompts x , π_{θ} denotes our policy, and $r(y | x)$ denotes whether y is a correct response to x . In this problem, we will derive the surrogate objectives optimized by our GRPO variants, where these surrogates take the form

$$\nabla_{\theta} J_{\theta, w} = \nabla_{\theta} \mathbb{E}_{x \sim \rho} \left[w(x, \text{stopgrad}(\pi_{\theta})) \mathbb{E}_{y \sim \pi_{\theta}} r(y | x) \right]. \quad (40)$$

for some reweighting over prompts w (where we do not differentiate through w).

(a) Recall that the Dr. GRPO policy gradient estimator is given by

$$\mathbb{E}_{x \sim \rho} \left[\frac{1}{Z} \sum_{j=1}^G (r(y^{(j)} | x) - \mu) \nabla_{\theta} \log \pi_{\theta}(y^{(j)} | x) \right], \quad (41)$$

where $\mu = \frac{1}{G} \sum_{j=1}^G r(y^{(j)} | x)$. Setting the constant normalizer $Z = G$ and taking group size $G \rightarrow \infty$, for what reweighting function w is this estimator equivalent to optimizing the surrogate objective $J_{\theta, w}$?

Deliverable: An expression in terms of a subset of the problem parameters, with a few sentences of justification.

(b) Assuming constant normalization, the GRPO estimator differs from Dr. GRPO in that it divides by the standard deviation:

$$\mathbb{E}_{x \sim \rho} \left[\frac{1}{Z} \sum_{j=1}^G \frac{r(y^{(j)} | x) - \mu}{\text{std}} \nabla_{\theta} \log \pi_{\theta}(y^{(j)} | x) \right]. \quad (42)$$

Setting the constant normalizer $Z = G$ and taking group size $G \rightarrow \infty$, for what reweighting function w is this estimator equivalent to optimizing the surrogate objective $J_{\theta, w}$?

Deliverable: An expression in terms of a subset of the problem parameters, with a few sentences of justification.

(c) MaxRL instead divides by the group mean, or

$$\mathbb{E}_{x \sim \rho} \left[\frac{1}{Z} \sum_{j=1}^G \frac{r(y^{(j)} | x) - \mu}{\mu} \nabla_{\theta} \log \pi_{\theta}(y^{(j)} | x) \right]. \quad (43)$$

Setting the constant normalizer $Z = G$ and taking group size $G \rightarrow \infty$, for what reweighting function w is this estimator equivalent to optimizing the surrogate objective $J_{\theta, w}$?

Deliverable: An expression in terms of a subset of the problem parameters, with a few sentences of justification.

答案

记 $\eta(x) = \mathbb{E}_{y \sim \pi_{\theta}}[r(y | x)]$ 。score-function 恒等式给出 $\nabla_{\theta} \eta(x) = \mathbb{E}[(r - \eta) \nabla_{\theta} \log \pi_{\theta}]$ 。 $Z = G$ 且 $G \rightarrow \infty$ 时，组内均值 $\mu \rightarrow \eta(x)$ ，标准差收到 $\sqrt{\text{Var}(r | x)}$ 。二元 reward 的方差是 $\eta(1 - \eta)$ 。

(a) Dr. GRPO 的极限是 $\mathbb{E}_x[\nabla_{\theta} \eta(x)]$ ，也就是 $w(x) = 1$ 。

(b) 再除以标准差后，梯度变成 $\mathbb{E}_x[\nabla_{\theta} \eta(x) / \sqrt{\eta(x)(1 - \eta(x))}]$ 。这是 $w(x) = 1 / \sqrt{\eta(x)(1 - \eta(x))}$ 且不对 w 求导。 η 靠近 0 或 1 时权重大，组内几乎全错或全对的题被放大。

(c) MaxRL 除以 $\mu \rightarrow \eta$ ，梯度是 $\mathbb{E}_x[\nabla_{\theta} \eta(x) / \eta(x)]$ 。对应 $w(x) = 1 / \eta(x)$ 。平均奖励越低的题权重越大。

题目 (compute_group_normalized_rewards_maxrl) : MaxRL group normalization (0.5 分) 题目来源: cs336_spring20

Problem (compute_group_normalized_rewards_maxrl): MaxRL Group normalization (0.5 points)

Deliverable: Update your method `compute_group_normalized_rewards` to support `advantage_normalizer = "mean"`. Like `advantage_normalizer = "std"`, you should add `advantage_eps` to the normalizer to avoid division by zero.

To test your code, implement `adapters.run_compute_group_normalized_rewards`. Then, run the test with:

```
uv run pytest -k compute_group_normalized_rewards_maxrl
```

and make sure your implementation passes it.

答案

`advantage_normalizer="mean"` 时，减去组内均值后再除以组内平均 reward 加 `advantage_eps`。 `test_compute_group_norm` 通过。

题目 (think_about_advantage_normalization) : 思考 advantage normalization (2 分) 题目来源: cs336_spring2026_as

Think about advantage normalization (2 points)

Before running any experiments, think about the difference between normalizing the group advantages by the group std, the group mean, or doing no advantage normalization. What are the pros and cons of each approach? Are there specific settings or examples where one approach seems better?

Deliverable: A few sentences of discussion.

答案

除以组内标准差 (GRPO, `baseline="mean"`) : 不同题的 advantage 尺度更接近。但若组内 reward 全相同，减均值后 advantage 先全是 0，再除以 $\sigma + \epsilon$ 仍是 0——这是无信号，不是被 ϵ 放大。真正会「分母 $\rightarrow 0$ 而分子非 0」的是 `baseline="none"` 再除以 std: 例如二元 reward 组内全对，raw advantage 全为 1, $\sigma = 0$ ，会得到 $\approx 1/\epsilon$ 的巨大权重。

除以组内均值 (MaxRL, `baseline="mean"`) : 实现是 $(r - \mu) / (\mu + \epsilon)$ ，方向仍是抬高低 μ 的难题。全错或全对时分子为 0，advantage 仍是 0， ϵ 不会放大这些组。 $G = 8$, $\epsilon = 10^{-6}$ 且恰有一条正确时， $\mu = 1/8$ ，正确样本约 $0.875/0.125 = 7$ ， ϵ 只带来约 10^{-5} 的相对变化；错误样本约 -1。同一组用标准差作分母时，正确样本大约是 2.5。MaxRL 更大，是因为分母是 μ 而不是 std，量级约 $G - 1$ 。5.3 的 $w(x) = 1/\eta(x)$ 是 $G \rightarrow \infty$ 时的题目权重，不是 ϵ 在单组上的上限。

不归一化 (Dr. GRPO / RFT 的 advantage 部分) : 尺度直接跟着 reward 与 baseline 走；配合 constant loss 时长回答 token 多、总梯度更大。二元 0/1、组内常全错的 GSM8K 上，我倾向不除以 std: mean

baseline 下大量同分组的 std 归一化并没有带来对比信号，反而在 mixed 组里引入额外的难度 reweighting（见 5.3 推导）。

5 RL 算法变体

5.4 实验

题目 (grpo_train_step_variants_on_policy) : GRPO train-step 变体 (2.5 分) 题目来源: cs336_spring2026_assignment5

GRPO train step variants (2.5 points)

Deliverable: Update your grpo_train_step method to support the full range of on-policy variants (still with importance_reweighting_method = "none"). These options include baseline:

Literal["mean", "none"], advantage_normalizer: Literal["std", "none", "mean"], and loss_normalization: Literal["sequence", "constant"]. Also, to speed up training, your method should avoid passing zero advantage sequences into the model.

Note that for baseline = "none", the incorrect samples get zero reward and therefore zero weight in the loss, so it is unnecessary to pass them through the model. You can take advantage of this fact to adjust the implementation and speed up training.

To test your code, implement adapters.run_grpo_train_step. Then run `uv run pytest -k test_grpo_train_step_variants_on_policy` and confirm it passes.

答案

grpo_train_step 已支持完整 on-policy 变体；在固定 rollout batch 布局下，于每个 microbatch 内跳过 advantage == 0 的 sequence（不先把 batch 缩成任意长度，避免 batch_size // grad_accum 为 0），sequence normalization 时用原始 microbatch 的 sequence 数作分母以保持 gradient 与不剪枝版一致。若整步没有任何非零 advantage 序列，仍把梯度写成 0 再调用 optimizer.step()，使 Adam 动量按零梯度衰减。test_grpo_train_step_variants_on_policy 四个组合均通过。训练入口见 scripts/train_grpo_variants.py 的 --recipe。

题目 (grpo_experiments_variants_on_policy) : 比较不同 RL 算法的性能 (约 8 B200 小时, 10 分)
题目来源: cs336_spring2026_assignment5_alignment.pdf

Compare the performance of different RL algorithms (8 B200 hrs) (10 points)

Keeping hyperparameters fixed with respect to your standard GRPO training runs (for your choice of learning rate), run the following variations, with 4 random seeds each. You should use the zero-shot r1_zero prompt.

- GRPO_constant: standard GRPO, but doing constant normalization for loss aggregation, rather than sequence normalization.
- Dr_GRPO: GRPO_constant but with advantage_normalizer = "none".
- RFT: GRPO_constant but with advantage_normalizer = "none" and baseline = "none". Note that for this method, there is no need to pass incorrect samples through the model being trained, which should speed up training.
- MaxRL: GRPO_constant but with advantage_normalizer = "mean". Note that the original MaxRL implementation normalizes the loss by the number of tokens per microbatch, but we will use constant normalization instead.

Compared to the standard GRPO runs you ran previously, how do these methods perform?

Which methods have the best average performance, and which methods have the lowest variance? Are there methods that you think would benefit from more hyperparameter tuning?

Based on the variance between runs, how confident are you in your findings?

Deliverable: Commentary, supported by plots for referenced metrics.

答案

学习率仍用 1×10^{-5} ，prompt 仍是 zero-shot r1_zero，其余超参与标准 GRPO 相同。四种变体各 4 个 seed，和 results/standard_seed_* 比较。

方法	四个 seed 的最终验证答案奖励	平均	样本标准差
标准 GRPO	47.9%, 45.6%, 12.3%, 46.7%	38.1%	17.2 pp
GRPO constant	49.9%, 51.5%, 49.4%, 47.7%	49.6%	1.6 pp
Dr. GRPO	48.4%, 49.3%, 49.6%, 49.4%	49.2%	0.5 pp
MaxRL	50.6%, 49.2%, 44.8%, 45.9%	47.6%	2.7 pp
RFT	47.5%, 44.8%, 48.6%, 46.5%	46.9%	1.6 pp

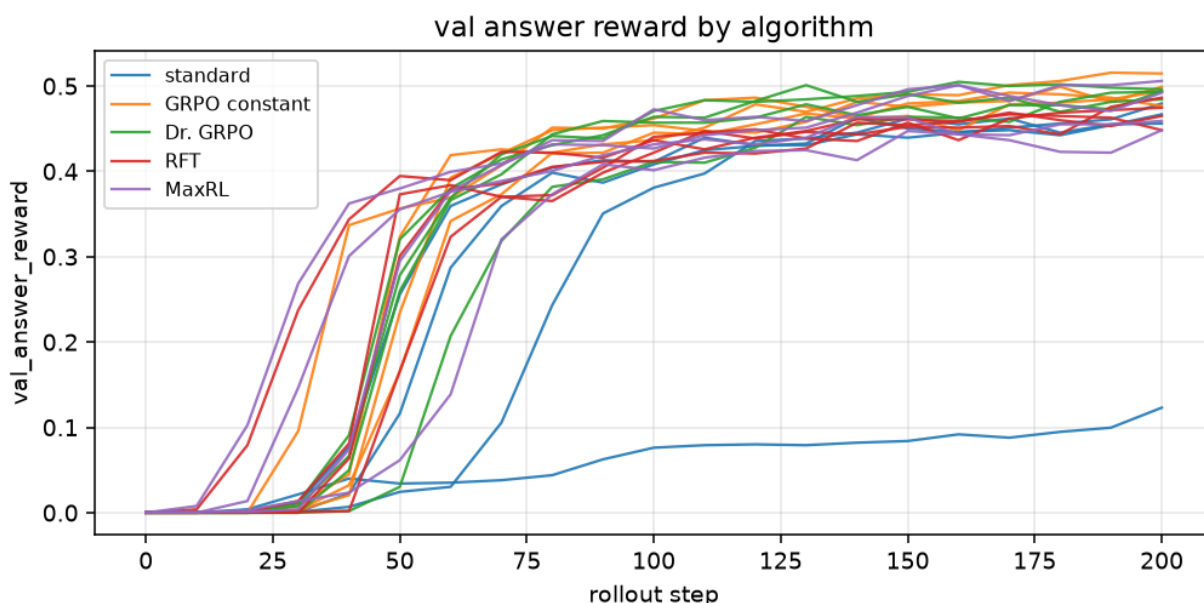


Figure 14: 不同算法的验证答案奖励

平均最高的是 GRPO constant, 49.6%。Dr. GRPO 只低 0.4 个百分点, 而它的样本标准差只有 0.5 个百分点, 是波动最小的。四种 constant normalization 的 16 次 run 全部落在 44.8%–51.5%, 没有再出现标准 GRPO 那种掉到 12.3% 的 seed。最差的 constant run (47.7%) 已经和标准 GRPO 最好的 seed (47.9%) 持平。

这个差距比四种变体彼此之间的差距更清楚。GRPO constant 和 Dr. GRPO 的差别小于它们自己的标准差, 不能据此说平均最低, 但四次都在 44.8%–48.6%, 仍然高于标准 GRPO 的平均。MaxRL 的波动比另外三种变体大一些。

格式奖励最后都在 84% 以上, 方法之间没有拉开。长度上, 标准 GRPO 失败的那次缩到 30 token; 成功 run 大约 130–160。GRPO constant 和 Dr. GRPO 收在 143–176, seed 之间更齐。RFT 最长, 也最散, 从 145 到 238。MaxRL 在 146–190。

这张 entropy 图不能和标准 GRPO 比高低。标准实验把该步全部回答 token 算进平均, 整组 advantage 为 0 的步也有大约 1 的 entropy。变体用的是现在的 grpo_train_step: 只平均非零 advantage 序列; 整步都没有这种序列时, 日志写成 0 并标 token_entropy_skipped。图上前几十步贴在 0, 是这些跳过的步, 不是测到的 entropy。因此不能说变体比标准 GRPO 更确定, 也不能用「失败 seed 停在 0.33、其他方法没有」来比较。

标准 GRPO 最值得再调学习率。它是唯一出现短回答崩溃的方法, 而 5×10^{-6} 已经表明把学习率降低可以去掉这种失败。长度跨了将近 100 token, 如果在意回答会不会越写越长, 也值得单独看长度归一化。GRPO constant 和 Dr. GRPO 在 1×10^{-5} 这一个学习率下, 四次结果挤在两个百分点以内。这只能说明这个学习率上四次很齐, 不能推出

在这次固定超参的 16 次 run 里, 没有一次落到标准 GRPO 那个 12.3% 的 seed。标准 GRPO 的平均 38.1% 被这一次拉低, 所以「高于这个平均」主要反映的是没有再出现那次失败。GRPO constant 和 Dr. GRPO 谁更高, 以及 RFT 是否真的低于它们, 现有波动盖不住这几个百分点的差别。

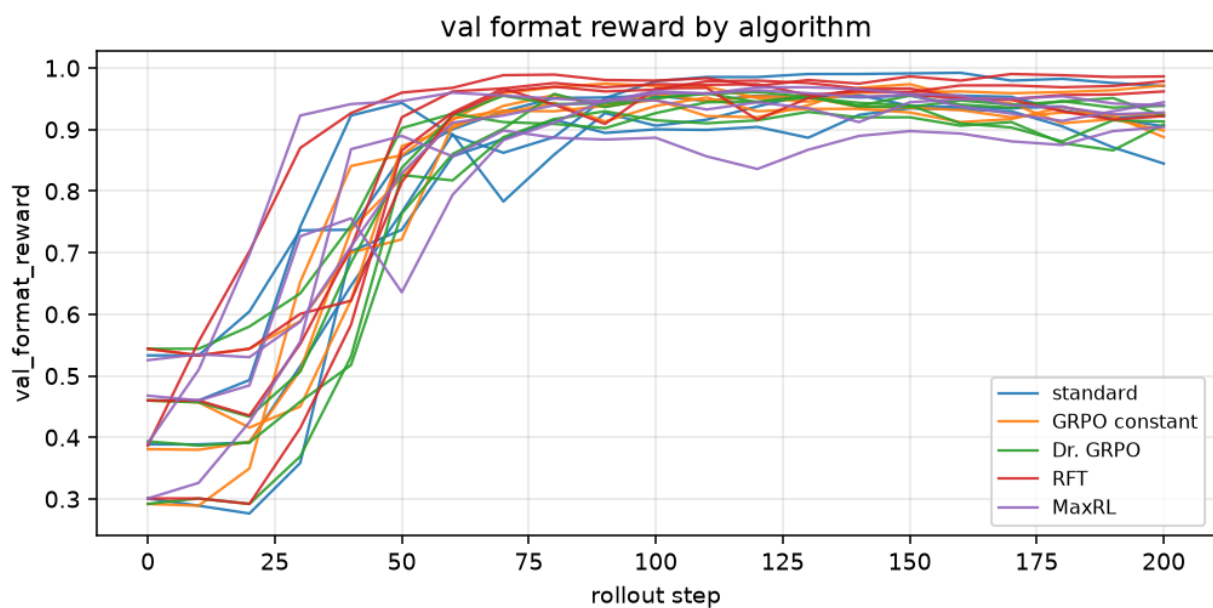


Figure 15: 不同算法的验证格式奖励

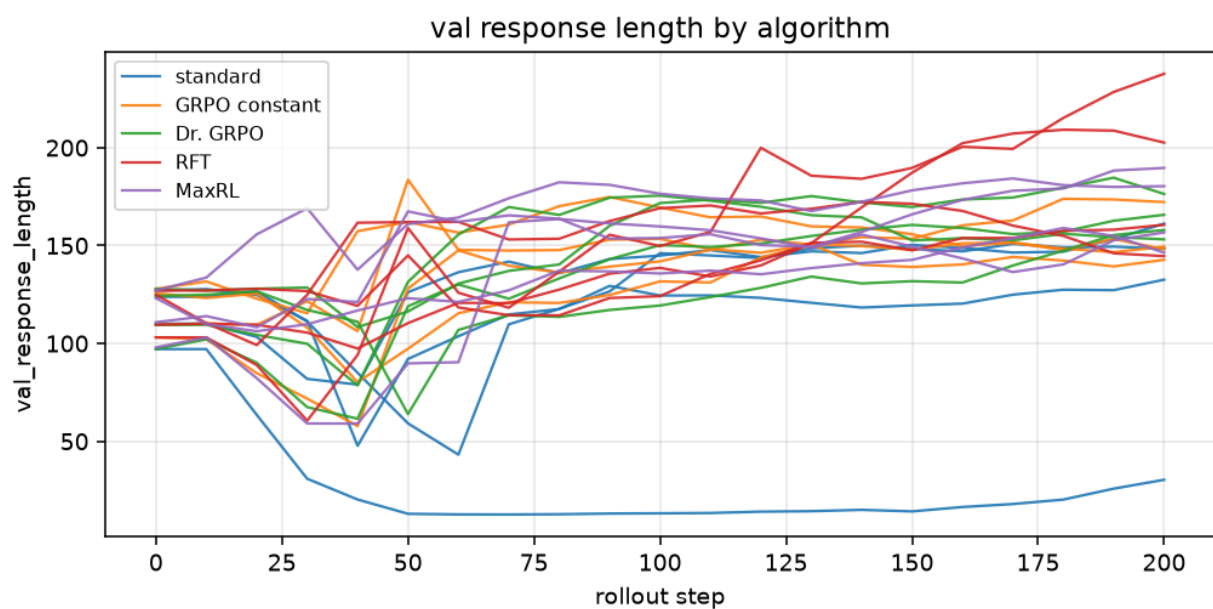


Figure 16: 不同算法的验证回答长度

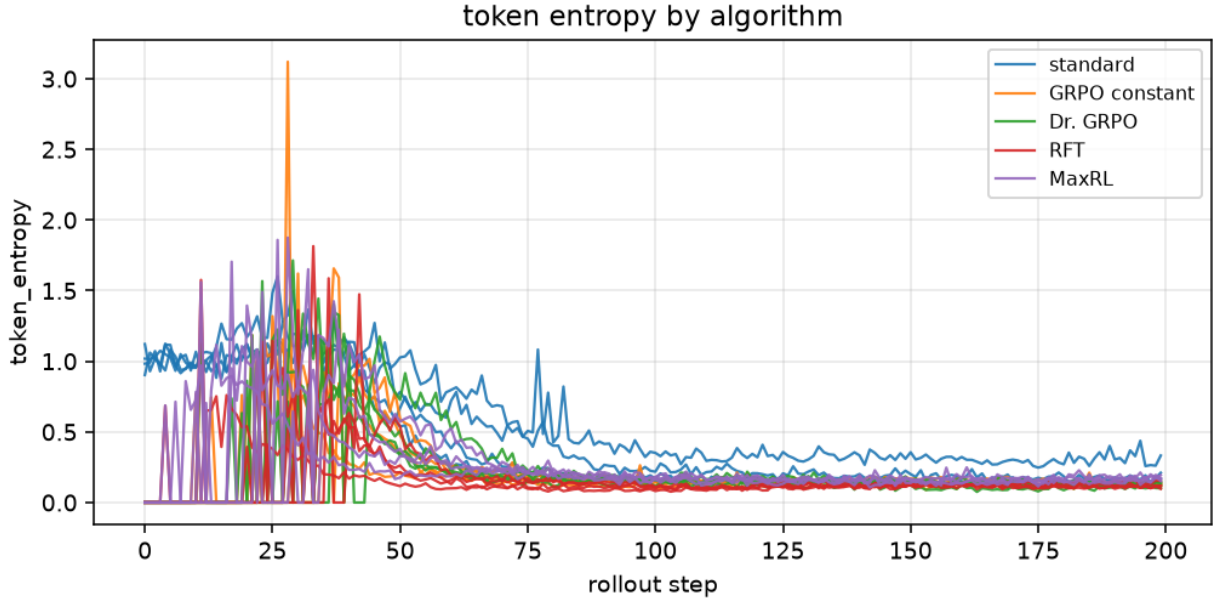


Figure 17: 不同算法的 token entropy

6 Off-policy RL

6.2 PPO/GRPO 风格的 importance reweighting 与 clipping

题目 (derive_surrogate_objectives) : 推导 importance reweighting 方法的 surrogate objective (2 分)

题目来源: cs336_spring2026_assignment5_alignment.pdf

Derive surrogate objectives for importance reweighting approaches (2 points)

Above, we saw that token-level importance reweighting optimizes expected reward under surrogate policies where we sample from the old policy for all but one position. Suppose we instead consider a “pairwise” importance reweighting approach given by the following policy gradient estimator:

$\sum$

L

$2t-1 \pi_\theta(y_{2t-1} | x, y_{<2t-1}) \pi_\theta(y_{2t} | x, y_{<2t}) \pi_0(y_{2t-1} | x, y_{<2t-1}) \pi_0(y_{2t} | x, y_{<2t}) r(y | x) \nabla \log(\pi_\theta(y_{2t-1} | x, y_{<2t-1}) \pi_\theta(y_{2t} | x, y_{<2t}))$ where π_θ is our current policy, π_0 is our stale sampling policy, and $y \sim \pi_0(y | x)$. What surrogate objective is this estimator optimizing?

Deliverable: An expression in terms of a subset of the problem parameters, accompanied by a derivation.

答案

$$\sum_{t=1}^{L/2} \frac{\pi_\theta(y_{2t-1} | x, y_{<2t-1}) \pi_\theta(y_{2t} | x, y_{<2t})}{\pi_0(y_{2t-1} | x, y_{<2t-1}) \pi_0(y_{2t} | x, y_{<2t})} r(y | x) \nabla_\theta [\log(\pi_\theta(y_{2t-1} | x, y_{<2t-1}) \pi_\theta(y_{2t} | x, y_{<2t}))]. \quad (55)$$

其中:

- π_θ 是当前 policy;
- π_0 是 stale sampling policy;
- $y \sim \pi_0(y | x)$.

这个 estimator 优化的 surrogate objective 是什么?

记第 t 对 token 的位置为 $\{2t-1, 2t\}$ 。定义 surrogate policy $\tilde{\pi}_t$: 这两个位置由当前 policy π_θ 采样, 其余位置由旧 policy π_0 采样。它相对 π_0 的 importance weight 正好是题目里那两个 ratio 的乘积, 而

$$\nabla_{\theta} \log \tilde{\pi}_t(y | x) = \nabla_{\theta} \log \pi_{\theta}(y_{2t-1} | x, y_{<2t-1}) + \nabla_{\theta} \log \pi_{\theta}(y_{2t} | x, y_{<2t}).$$

按讲义式(51)的写法，式(55)优化的目标是这些 surrogate policy 下的期望奖励之和。它是标量目标，不是梯度：

$$J_{\text{pair}}(x) = \sum_{t=1}^{L/2} \mathbb{E}_{y \sim \tilde{\pi}_t(y|x)} [r(y | x)].$$

用 importance sampling 改到 $y \sim \pi_0$ ，目标本身仍然等于

$$J_{\text{pair}}(x) = \sum_{t=1}^{L/2} \mathbb{E}_{y \sim \pi_0(y|x)} \left[\frac{\pi_{\theta}(y_{2t-1} | x, y_{<2t-1}) \pi_{\theta}(y_{2t} | x, y_{<2t})}{\pi_0(y_{2t-1} | x, y_{<2t-1}) \pi_0(y_{2t} | x, y_{<2t})} r(y | x) \right].$$

再对 θ 求导，并用 log-derivative，才得到题目中的 estimator。右边带的是 score，不是 $\mathbb{E}[r]$ ：

$$\nabla_{\theta} J_{\text{pair}}(x) = \sum_{t=1}^{L/2} \mathbb{E}_{y \sim \pi_0(y|x)} \left[\frac{\pi_{\theta}(y_{2t-1} | x, y_{<2t-1}) \pi_{\theta}(y_{2t} | x, y_{<2t})}{\pi_0(y_{2t-1} | x, y_{<2t-1}) \pi_0(y_{2t} | x, y_{<2t})} r(y | x) \nabla_{\theta} \log (\pi_{\theta}(y_{2t-1} | x, y_{<2t-1}) \pi_{\theta}(y_{2t} | x, y_{<2t})) \right].$$

(对 x 再按 prompt 分布 ρ 取期望即全数据集 objective。)

题目 (compute_policy_gradient_loss_off_policy)：使用 token-level reweighting 的 off-policy policy gradient (1 分) 题目来源：cs336_spring2026_assignment5_alignment.pdf

Off-policy policy gradient with tokenlevel reweighting (1 point)

Deliverable: Update your method compute_policy_gradient_loss to support importance_reweighting_method = “noclip” or “grpo”, which use the old_log_probs and cliprange arguments. old_log_probs denotes the per-token log probabilities under the model used to generate the rollouts, and you’ll have to add a few lines to your training script to compute these. cliprange denotes the clipping strength parameter ε .

To test your code, implement adapters.run_compute_policy_gradient_loss . Then run uv run pytest -k test_compute_policy_gradient_loss_off_policy and ensure the test passes.

Note: while clipping is motivated in PPO as a way to prevent the current policy from straying too far from the old policy, it can also be viewed as a way to reduce the variance of the importance reweighted estimator, since we are squashing importance weight terms that are too large. If we are less concerned with straying far from the old policy and want to be more aggressive in our updates, a much simpler and more direct way to control the variance is to just upper-bound-clip the importance weight in the gradient estimator directly:

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i,j} \frac{1}{\text{len}(y^{(i,j)})} \sum_t \min(w_t^{(i,j)}, 1 + \varepsilon) A^{(i,j)} \nabla_{\theta} \log \pi_{\theta}(y_t | x, y_{<t}). \quad (61)$$

where unlike PPO/GRPO we still take non-zero gradient through actions that are upweighted by more than $1 + \varepsilon$. This simpler clipping procedure is proposed in CISPO [MiniMax et al., 2025] (note: they also normalize by per-group token count instead of per-sequence length, but we write the sequence-normalized objective for simplicity). Optionally, you’re welcome to try out CISPO and compare it to the other clipping methods.

答案

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i,j} \frac{1}{\text{len}(y^{(i,j)})} \sum_t \min(w_t^{(i,j)}, 1 + \varepsilon) A^{(i,j)} \nabla_{\theta} \log \pi_{\theta}(y_t | x, y_{<t}). \quad (61)$$

这里与 PPO/GRPO 的区别是：即使某个 positive-advantage action 已经被当前 policy 提高了超过 $1 + \varepsilon$ 倍，它仍然会保留非零 gradient，只是 importance weight 本身被截断。

这种更简单的 clipping 方法来自 CISPO [MiniMax et al., 2025]。原论文还会按 group 的 token count 进行 normalization，而不是按每条 sequence length；这里为了简化，仅写成 sequence-normalized objective。

该方法是可选内容，你也可以尝试 CISPO 并与其他 clipping 方法比较。

noclip 的逐 token loss 是 $-(Aw_t)$ ，其中 $w_t = \exp(\log \pi_\theta - \log \pi_0)$ 。grpo 用 PPO 的 $\min(Aw_t, A\text{clip}(w_t, 1 - \epsilon, 1 + \epsilon))$ ，再取负。metadata 里的 clip_fraction 是 response token 中 ratio 落在 $[1 - \epsilon, 1 + \epsilon]$ 之外的比例。它不等于梯度被截成 0 的比例：正 advantage 且 $w < 1 - \epsilon$ ，或负 advantage 且 $w > 1 + \epsilon$ 时，min 仍走未截断的一支。test_compute_policy_gradient_loss_off_policy 通过。上面的 CISPO 只截断正 advantage 的权重，这次没有实现。

6 Off-policy RL

6.3 GSPO

题目 (think_about_importance_reweighting)：思考 importance reweighting (2 分) 题目来源：cs336_spring2026_assignment5

Think about importance reweighting (2 points)

Consider three possible importance reweighting strategies for off-policy RL: (a) no importance reweighting, (b) PPO/GRPO-style clipped token-level importance reweighting, and (c) GSPO-style clipped sequence-level importance reweighting using the geometric mean. In terms of trading off bias and variance, where does each approach lie on the spectrum? Can you think of situations where one approach might intuitively be better than the others?

Deliverable: A few sentences of commentary.

答案

下面只比较题目给出的三种策略（不重加权 / PPO-GRPO token 级 clip / GSPO 几何平均 + clip）。

样本都来自旧 policy π_0 。要对准的目标是 $\nabla \mathbb{E}_{y \sim \pi_\theta} [r]$ ，或者式 (51) 那种 surrogate 期望。 π_0 不依赖 θ 时， $\nabla \mathbb{E}_{y \sim \pi_0} [r] = 0$ ，不能拿它当基准。三种权重也要分开：没有 ratio、单个 token ratio w_t 、联合比 $\prod_s w_s$ 、几何平均 $\exp(\text{mean}_t \log w_t) = (\prod_t w_t)^{1/\text{len}(y)}$ 。

策略	裁剪前对准什么	权重	偏差	方差
(a) 不重加权	只在 $\pi_\theta = \pi_0$ 时， $r \nabla \log \pi_\theta$ 的抽样期望对准 $\nabla \mathbb{E}_{y \sim \pi_\theta} [r]$	无	policy 拉开后，抽样分布不重合	没有 IS 这项额外方差最小
(b) token 级，裁剪前	对准的是式 (51)：只有第 t 个 token 来自 π_θ 。它不是联合 IS，因此一般不对准 $\nabla \mathbb{E}_{y \sim \pi_\theta} [r]$	每个位置一个 w_t ，不是 $\prod_s w_s$	prefix 和 suffix 都来自 π_0 ；clip 再在优势符号对应的 0	低于联合连乘，因为没有把 $\text{len}(y)$ 个 ratio 乘起来再压这一侧的尾部
(c) GSPO 几何平均，裁剪前	同样不是联合 IS。联合比是 $\prod_t w_t$ ；几何平均多了 $1/\text{len}(y)$ 次方	$\exp(\text{mean}_t \log w_t)$	相对联合 IS 有偏；这个次方还会再加一层	低于联合连乘；一个序列只有一个偏；这个次方还会在梯度里被平均成接近 1

直觉：(a) 适合 32 次更新的前几步、policy 仍接近采样 policy；(b) 用单 token ratio 换掉联合连乘，方差更低，代价是 surrogate 偏差；(c) 在 (b) 和完整联合比之间，回答很长时比连乘稳，但不是联合比的无偏替代。clip 比较的是裁掉哪一侧的尾部，不是三种权重谁更接近 π_0 下的期望。

题目 (compute_policy_gradient_loss_off_policy_gspo)：使用 sequence-level reweighting 的 off-policy policy gradient (1 分) 题目来源：cs336_spring2026_assignment5_alignment.pdf

Off-policy policy gradient with sequence-level reweighting (1 point)

Deliverable: Update your method `compute_policy_gradient_loss` to support `importance_reweighting_method = "gspo"`, which uses the `old_log_probs` and `cliprange` arguments. `old_log_probs` denotes the per-token log probabilities under the model used to generate the rollouts, and you'll have to add a few lines to your training script to compute these. `cliprange` denotes the clipping strength parameter ϵ . As above, return the negative objective so that minimizing the loss performs gradient ascent.

To test your code, implement `adapters.run_compute_policy_gradient_loss`. Then run `uv run pytest -k test_compute_policy_gradient_loss_off_policy_gspo` and ensure the test passes.

答案

`gspo` 在 response token 上对 $\log \pi_\theta - \log \pi_0$ 取平均再取指数，得到序列级 ratio，然后用与 GRPO 相同的 clip 和 min，并把这个标量写到该序列的每个位置。几何平均用对数平均实现，避免连乘溢出。`test_compute_policy_g` 通过。

6 Off-policy RL

6.4 实验

题目 (`grpo_train_step_off_policy`) : Off-policy GRPO train step (2.5 分) 题目来源: `cs336_spring2026_assignment5_align`

Off-policy GRPO train step (2.5 points)

Deliverable: Update your `grpo_train_step` method to support the off-policy arguments

(`importance_reweighting_method`, `old_log_probs`, `cliprange`). The function should now support the full range of arguments.

To test your code, implement `adapters.run_grpo_train_step`. Then run `uv run pytest -k test_grpo_train_step_off_policy` and confirm it passes.

答案

`grpo_train_step` 会把 `old_log_probs`、`cliprange` 和 `importance_reweighting_method` 传给 `compute_policy_gradient_loss`，并传 `clip_fraction`。`test_grpo_train_step_off_policy` 通过。`scripts/train_grpo_variants.py` 在 `--recipe offpolicy_*` 下对每个 inference batch 做 32 次 update (`train_batch_size=8`)，并在 rollout 后用 `compute_old_log_probs` 固定采样 policy 的 log-prob。

题目 (`grpo_experiments_off_policy`) : 比较不同 off-policy 算法的性能 (约 8 B200 小时, 10 分)

题目来源: `cs336_spring2026_assignment5_alignment.pdf`

Compare the performance of different off-policy algorithms (8 B200 hrs) (10 points)

Keeping hyperparameters fixed with respect to your standard GRPO training runs (for your choice of learning rate), run the following variations, with 4 random seeds each. You should use the zero-shot `r1_zero` prompt. If you would like, you can choose your favorite RL algorithm variant from the previous section instead of sticking with the standard version.

- `offpolicy_naive`: Instead of making inference and training batch size equal (both 256), reduce the training batch size to 1/32nd of the inference batch size ($256/32 = 8$). Remember to reduce the number of gradient accumulation steps by the same factor so we continue fully utilizing our GPUs. Keep `importance_reweighting_method = "none"`.
- `offpolicy_noclip`: `offpolicy_naive` with `importance_reweighting_method = "noclip"`.
- `offpolicy_clip`: `offpolicy_naive` with `importance_reweighting_method = "grpo"`.
- `offpolicy_gspo`: `offpolicy_naive` with `importance_reweighting_method = "gspo"`.

In addition to the metrics you're already logging, make sure to also log the clip fraction.

Compared to the fully on-policy GRPO runs you ran previously, how do these methods perform?

How does going off-policy affect training stability and variance between runs? How do the two clipping methods differ in terms of the clip fraction, and which one seems more stable? Are there methods that you think would

benefit from more hyperparameter tuning? Based on the variance between runs, how confident are you in your findings?

Deliverable: Commentary, supported by plots for referenced metrics.

答案

学习率仍是 1×10^{-5} ，prompt 仍是 zero-shot r1_zero，和标准 GRPO 相同。每个 rollout 有 256 条 response，拆成 32 次更新，每次 8 条，梯度累积步数是 1。四种方法各 4 个 seed。clip_fraction 记在 off-policy_clip 和 offpolicy_gspo 的每个训练步上。

方法	四个 seed 的最终验证答案奖励	平均	样本标准差
标准 GRPO	47.9%, 45.6%, 12.3%, 46.7%	38.1%	17.2 pp
off-policy naive	53.9%, 53.7%, 49.0%, 38.9%	48.9%	7.0 pp
noclip	50.2%, 54.2%, 55.0%, 52.6%	53.0%	2.1 pp
GRPO clip	53.8%, 54.7%, 54.6%, 50.1%	53.3%	2.2 pp
GSPO	51.9%, 47.9%, 52.9%, 48.6%	50.3%	2.4 pp

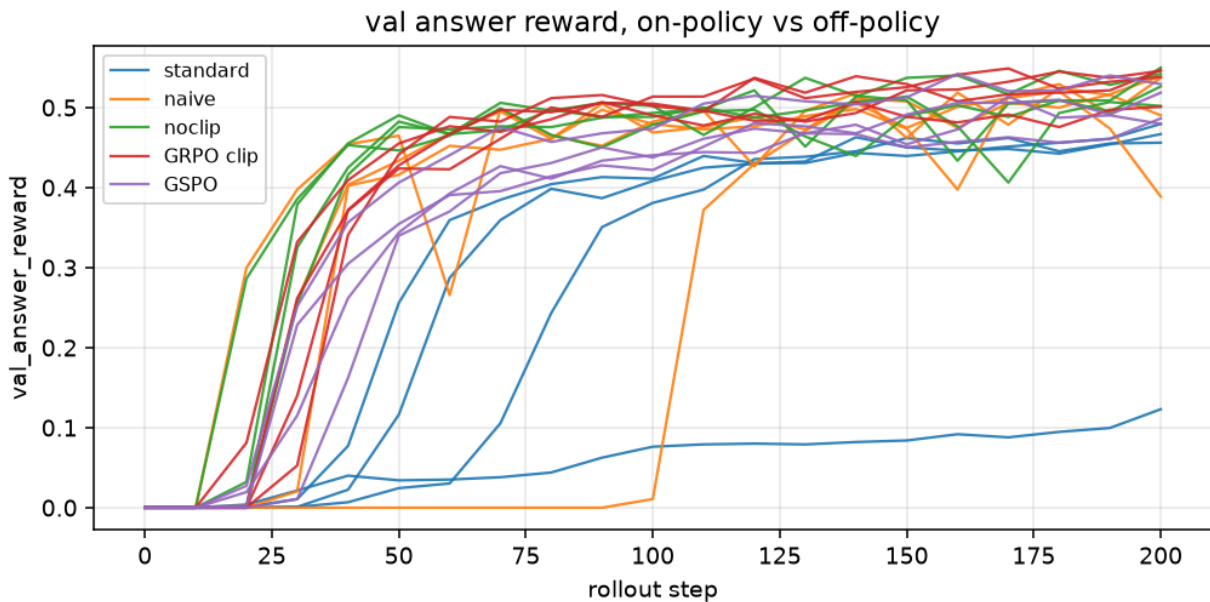


Figure 18: on-policy 与 off-policy 的验证答案奖励

只把一次大更新拆成 32 次小更新，不做 importance weighting，平均就已经从 38.1% 升到 48.9%，标准差从 17.2 降到 7.0 个百分点。加上不裁剪的 ratio 之后，四次都在 50.2%–55.0%，平均 53.0%。GRPO clip 和它几乎一样，平均 53.3%，标准差 2.2 个百分点。GSPO 平均 50.3%，略低，波动和 token 级 clip 差不多。四种 off-policy 都没有再出现标准 GRPO 那种掉到 12.3% 的 seed。naive 最弱的一次是 38.9%，其余三种的最低值是 GSPO 的 47.9%。

两种 clip 差在 ratio 越界的比例，不差在最后准不准。clip_fraction 记的是 response token 里 ratio 落在 $[1 - \varepsilon, 1 + \varepsilon]$ 外的比例，不是梯度被置 0 的比例，也不是序列条数的比例。正 advantage 且 ratio 低于 $1 - \varepsilon$ ，或负 advantage 且 ratio 高于 $1 + \varepsilon$ 时，PPO 的 min 仍用未截断的一支。GSPO 把同一个序列 ratio 铺到该序列的每个 token 上，所以它的 clip_fraction 是按回答长度加权的序列比例。

GRPO clip 的 cliprange 是 0.2，step 100 之后的 clip fraction 大约是 0.9%，四个 seed 都在 0.81%–0.90%。越界的 token 很少，曲线才和 noclip 重合。GSPO 的 cliprange 是 3×10^{-4} ，同一时段大约 55% 的 token 越界，四个 seed 在 52%–56%。这表示大约一半的 token 落在区间外，不能直接说成一半的序列条数被夹住。准确和 GRPO clip 一样，大约 2 个百分点，平均还低大约 3 个百分点。

这张 entropy 图同样不能用来下「off-policy 更低」的结论。标准曲线统计全部回答 token。off-policy 每一行是最多 32 次小更新的算术平均：每次只统计非零 advantage 序列；某次整组 advantage 为 0 时那次记 0。只有 32 次全部跳过，日志才标 token_entropy_skipped。图上前段的 0 是这些跳过步。后段不再标跳过，

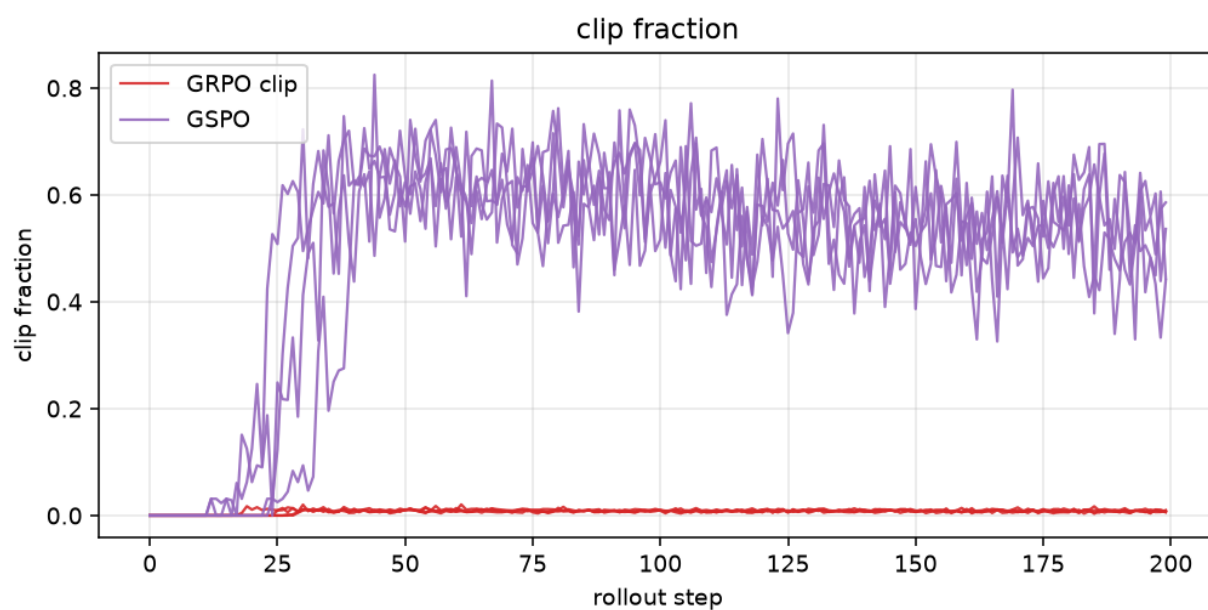


Figure 19: clip fraction

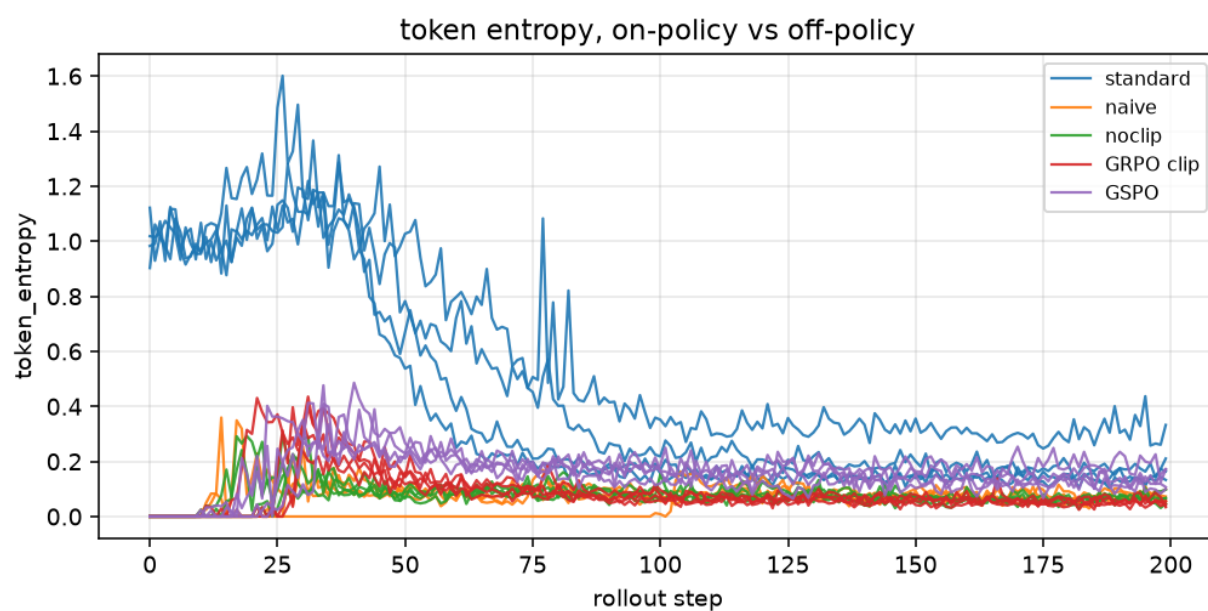


Figure 20: on-policy 与 off-policy 的 token entropy

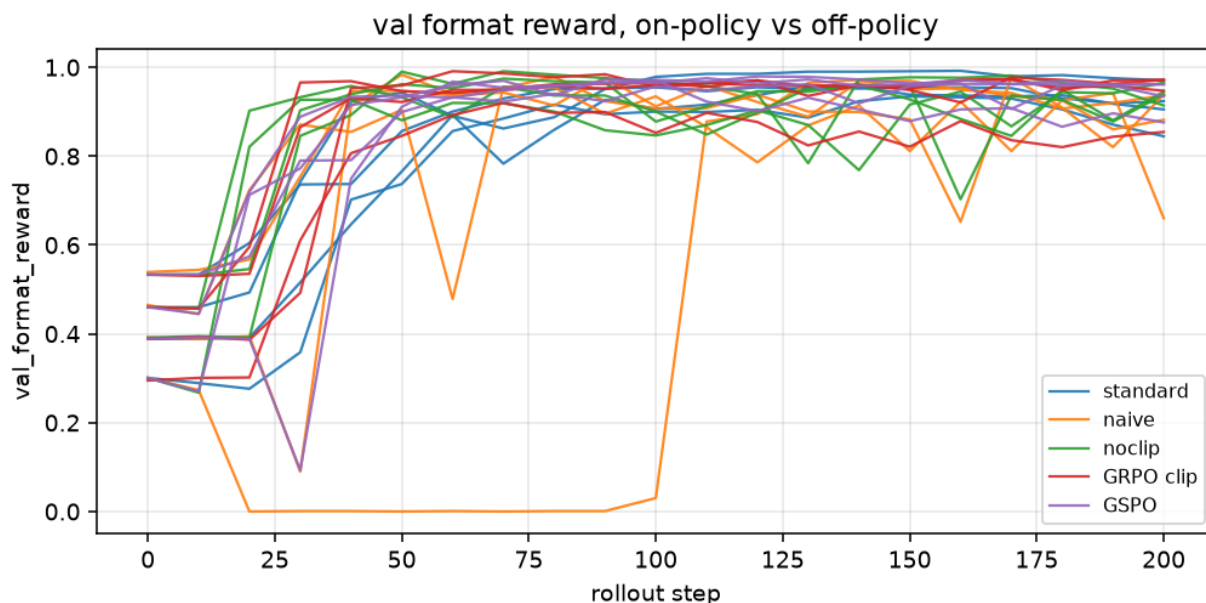


Figure 21: on-policy 与 off-policy 的验证格式奖励

entropy 的 0，而且分母和标准实验不是同一批 token。naive 有一个 seed 的格式奖励只有 66.0%，另外三种都在 85% 以上。回答长度没有再缩到 30 token 那种失败模式。

naive 最值得再调。它没有 importance weighting，四次里有一次明显掉队，标准差仍有 7 个百分点。GSPO 大约一半的 token 越界，区间可能偏紧；把 cliprange 放大会不会靠近 noclip，这次实验没有测。noclip 和 GRPO clip 在 1×10^{-5} 下四次都在 50%–55%。这只能说明这个学习率上四次很齐，不能推出再调超参平均奖励也不会差。

在这组固定超参的 run 里，带 ratio 的 12 次最低是 47.9%，高于标准 GRPO 的平均 38.1%，也高于它那个 12.3% 的 seed。标准平均被这一次拉低，所以这 12 次高于平均，说的是没有再出现那种失败，不是换学习率之后仍然和 GRPO clip 谁更高分不出来；token 级 clip 的越界比例大约 0.9%，和 noclip 的重合是一致的。GSPO 平均低大约 3 个百分点，和 2 个百分点的标准差比，还不能说它一定更差。

7 尝试你自己的 Policy-Gradient Estimator

题目 (try_your_own)：尝试你自己的 policy-gradient estimator (10 分) 题目来源: cs336_spring2026_assignment5_a

Try your own policy gradient estimator (10 points)

Now that you've seen a variety of policy gradient estimators that have been proposed in the literature, along with some of the theory behind them, it's time for you to propose your own!

Here are some suggestions:

- You could try a different advantage estimator, inducing a different reweighting over prompts.
- You could try a different importance reweighting strategy.
- You could try a different reward baseline in the advantage estimator.

Propose your own policy gradient estimator. Then, run it on the same RL setting of OLMo-2-0425-1B on GSM8K and compare it to the approaches we've already tried as baselines, using multiple random seeds and keeping problem parameters fixed to make the results comparable. Please change just one thing relative to the approaches tried so far (i.e. there should be at least one previous approach covered in this assignment such that your approach changes only one thing).

What is the theoretical justification or intuition behind your proposed change? Does your proposed change beat the baselines?

Deliverable: Commentary, supported by plots for referenced metrics, and optionally mathematical derivations if they are useful in justifying your approach.

答案

待填写。
